



微算機系統 2026 MICROPROCESSOR SYSTEMS

Instructor : Yen-Lin Chen(陳彥霖)

Dept. Computer Science and Information Engineering

National Taipei University of Technology

ylchen@ntut.edu.tw



處理器與時序電路的概念介紹

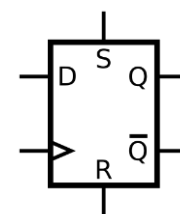
- 與組合電路不同，時序電路的輸出不僅取決於當前的輸入，還依賴於電路的前一個狀態，並依靠精確的時序脈波(CK)來協調各個功能單元的操作。

主要元件：RS正反器、D型正反器、JK正反器、T型正反器、暫存器

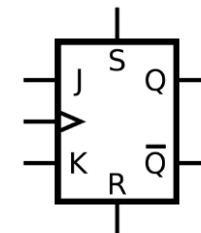
1. RS型正反器：最簡單，且也是組成其他正反器的基礎
2. D型正反器：為常見的正反器類型
3. JK型正反器：可實現多種狀態的轉換
4. T型正反器：常用於計數器
5. 暫存器：由多個正反器組成，可實現存儲多位數據

- 正反器的線路圖由邏輯閘組合而成，其結構均由SR閘鎖衍生而來

S\R的輸入會強迫Q輸出為1\0

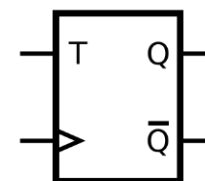


D型正反器



JK型正反器

將JK型的輸入併再一起構成T型



T型正反器



相關應用

1. 處理器中的暫存器

- CPU中的Register File由大量D型正反器構成
- RAX與EBX實際上是由32或64個D型正反器構成

2. 時序邏輯中的狀態機

- UART串列通訊控制器使用狀態機控制數據傳輸

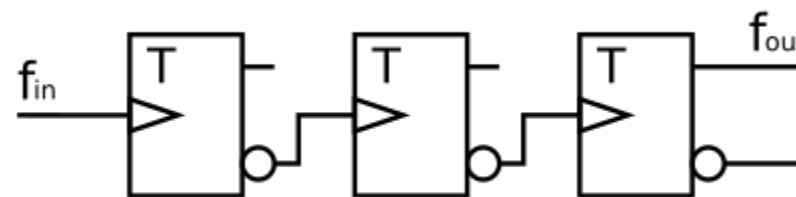
3. 記憶體元件

- SRAM：由多個觸發器構成
- DRAM：原理不同，但使用正反器控制時序

4. 分頻器

- 74系列邏輯IC中的74LS93是4位二進制計數器，內部使用多個正反器

5. 中斷控制同步電路



由3個T型實現的除8分頻器



目錄

- 基本門鎖器
- 閘控SR門鎖器
- 閘控D門鎖器
- 邊緣觸發D正反器
- T正反器
- JK正反器
- 術語總結
- 暫存器
- 計數器
- 重置同步
- 其他類型計數器
- 以CAD工具使用記憶體元件
- 以CAD工具使用暫存器和計數器



介紹-順序電路 (Sequential Circuit)

- 先前介紹過組合電路，每個輸出的值只跟輸入的訊號值有關。
- 有另一種邏輯函數其輸出值不只跟現在的輸入值有關，也跟過去的電路行為有關。這種電路包含**儲存元件**以儲存邏輯訊號的值。儲存元件的內容稱為電路的**狀態 (state)**。
- 當電路輸入值改變，新的輸入值會將電路留在相同的狀態，或使其改變到新的狀態。
- 輸入隨時間改變，電路會經歷一連串狀態。如此行為的電路稱為**順序電路** (sequential circuit)。
- 本課程將介紹可用作儲存元件的電路。



簡單的記憶體元件

- 圖1為基本的**記憶體元件**，由兩個反向器的迴圈組成。若假設 $A = 0$ ，則 $B = 1$ 。此電路將無限期保持這些值。我們稱此電路處於這些值所定義的狀態 (state)。

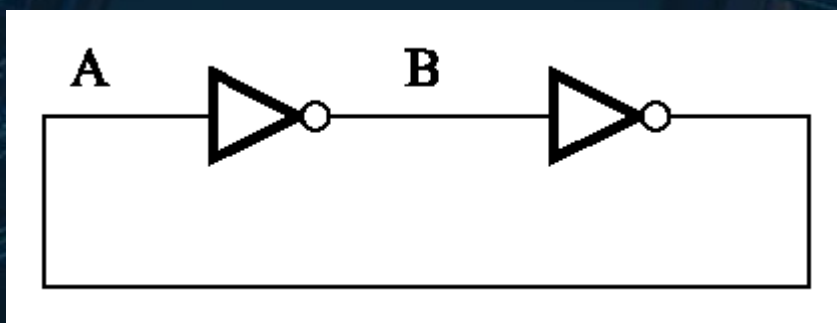


圖1 簡單的記憶體元件。



控制的記憶體元件

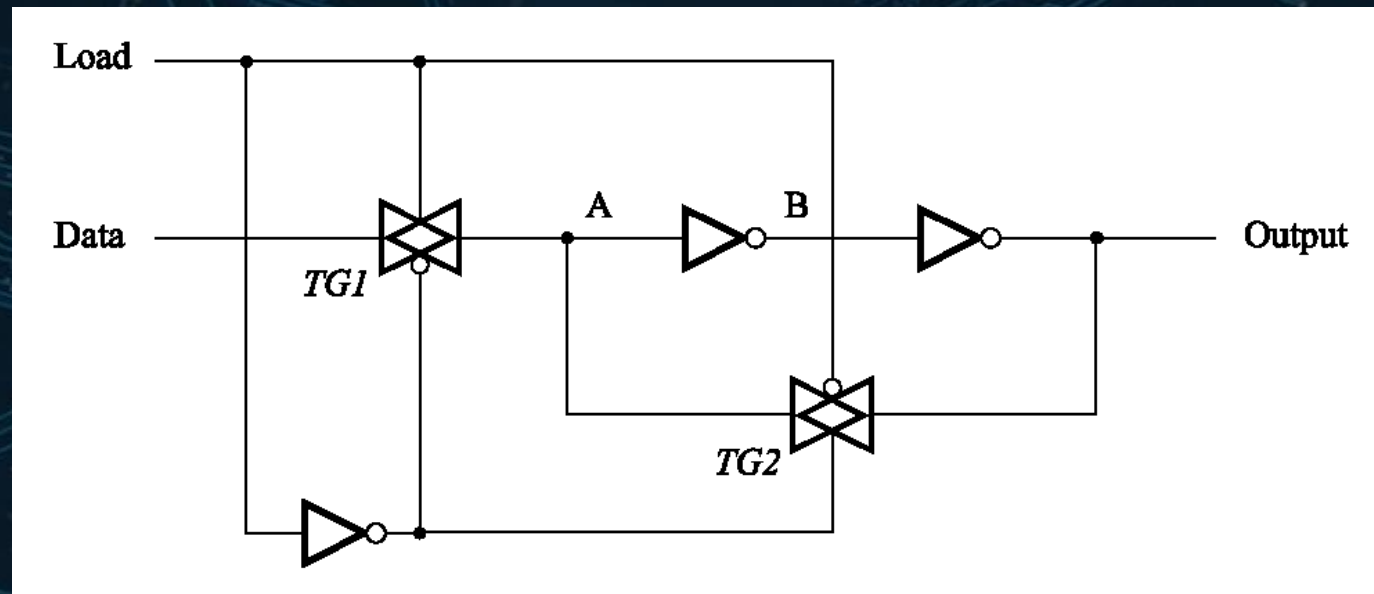


圖2 控制的記憶體元件。



記憶體發展趨勢

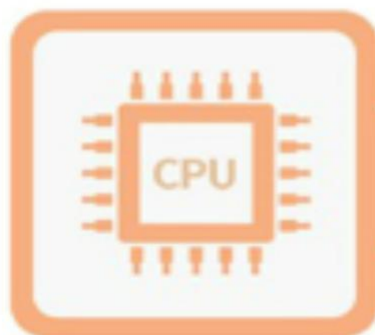
Today's Computer Chip

Von Neumann architectures

separation between program execution and data storage



Bottleneck Phenomenon

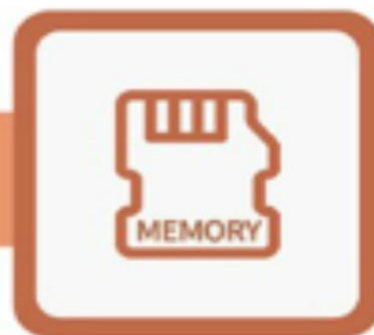


Central Processing Unit
(CPU)

匯流排



Bus
Transfer



Storage Unit
(Memory)

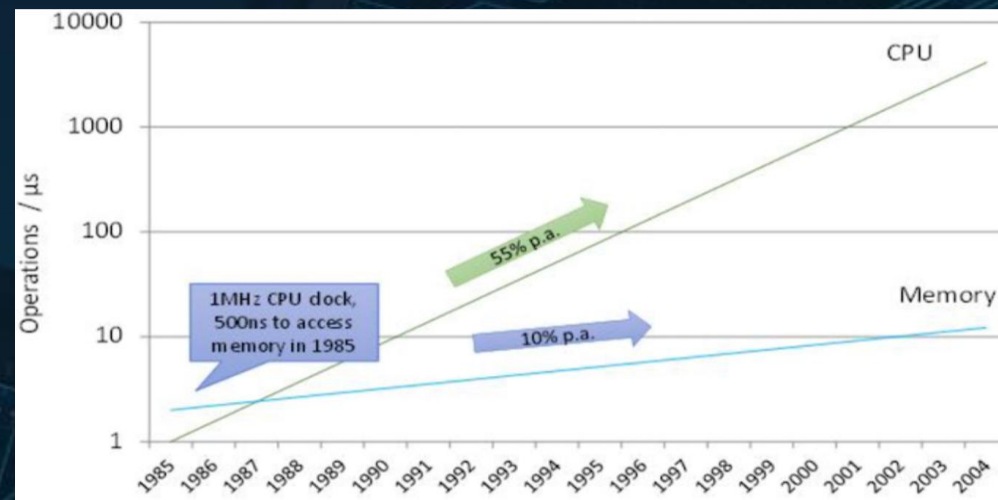
CPU操作速度
Fast

記憶體讀寫速率
Slow

Solution: Cache、HBM...

Memory Bandwidth:

是指從電腦系統記憶體中讀取或寫入資料的速率

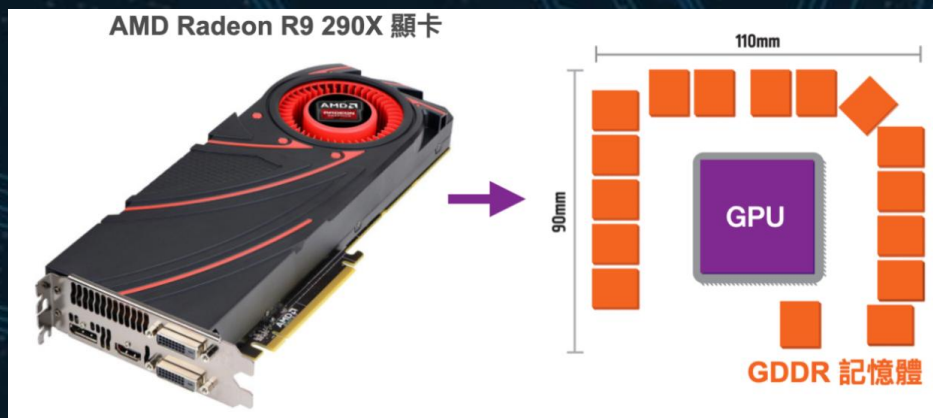


CPU的進步速度遠遠超過了記憶體



HBM (高頻寬記憶體)

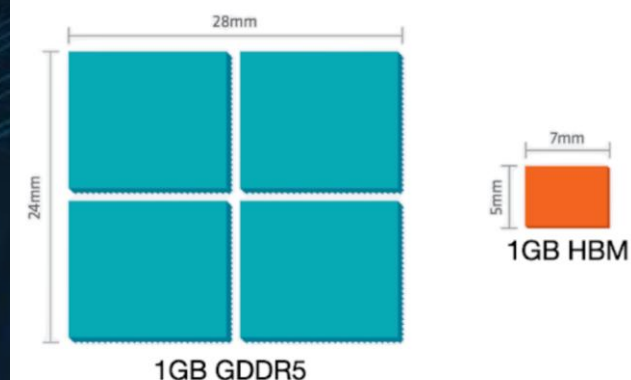
Traditional



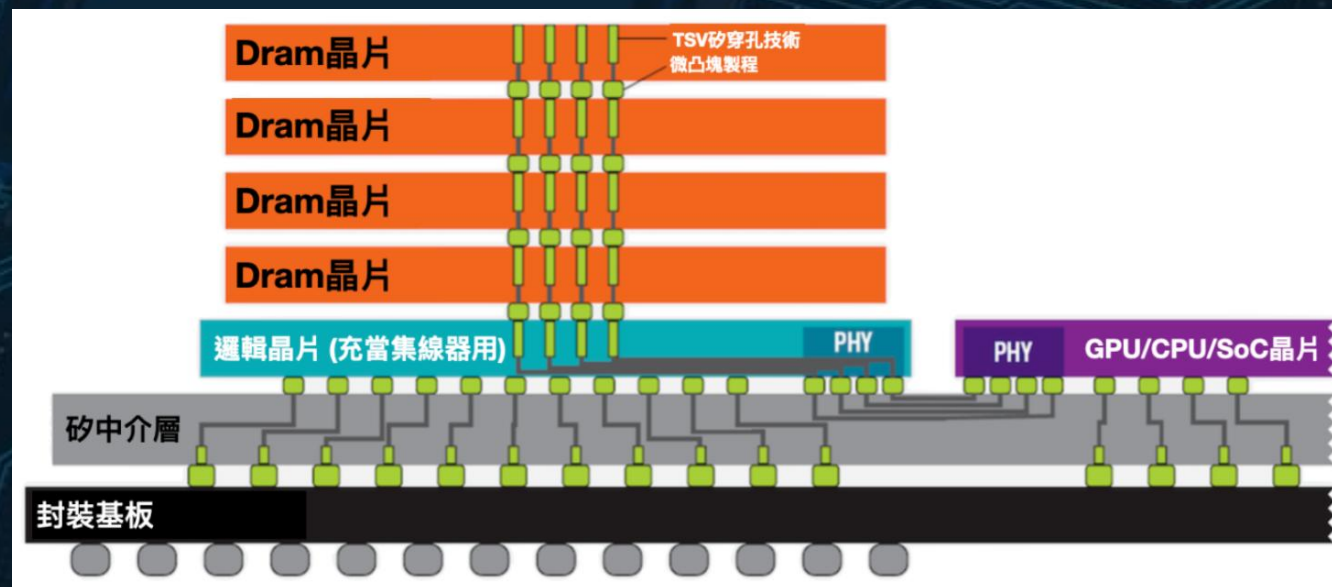
Cos: 大面積、拉線長

HBM vs GDDR5

大幅節省空間 (HBM 省下94%面積)



HBM

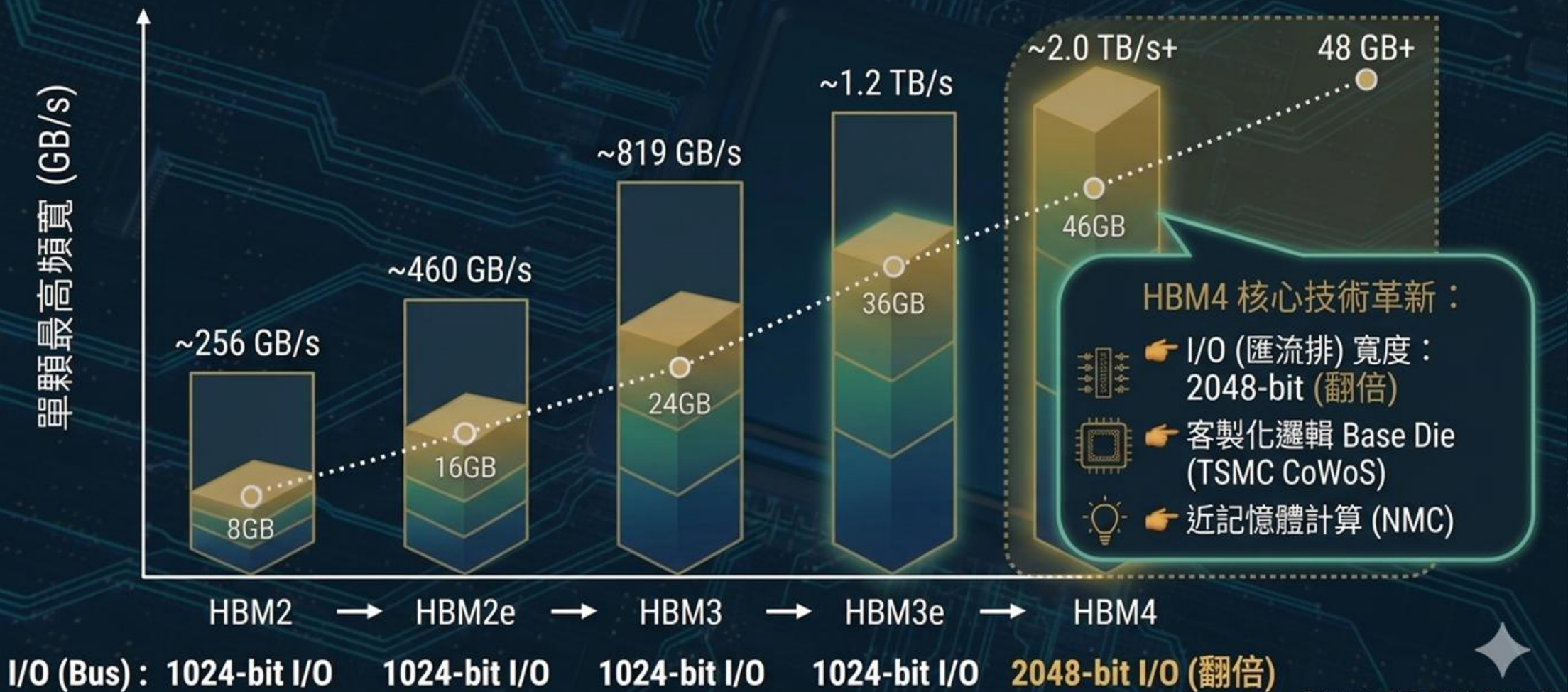


2.5D Package

大幅增加記憶體 I/O 寬度
減少資料傳輸距離 → 降低延遲與功耗



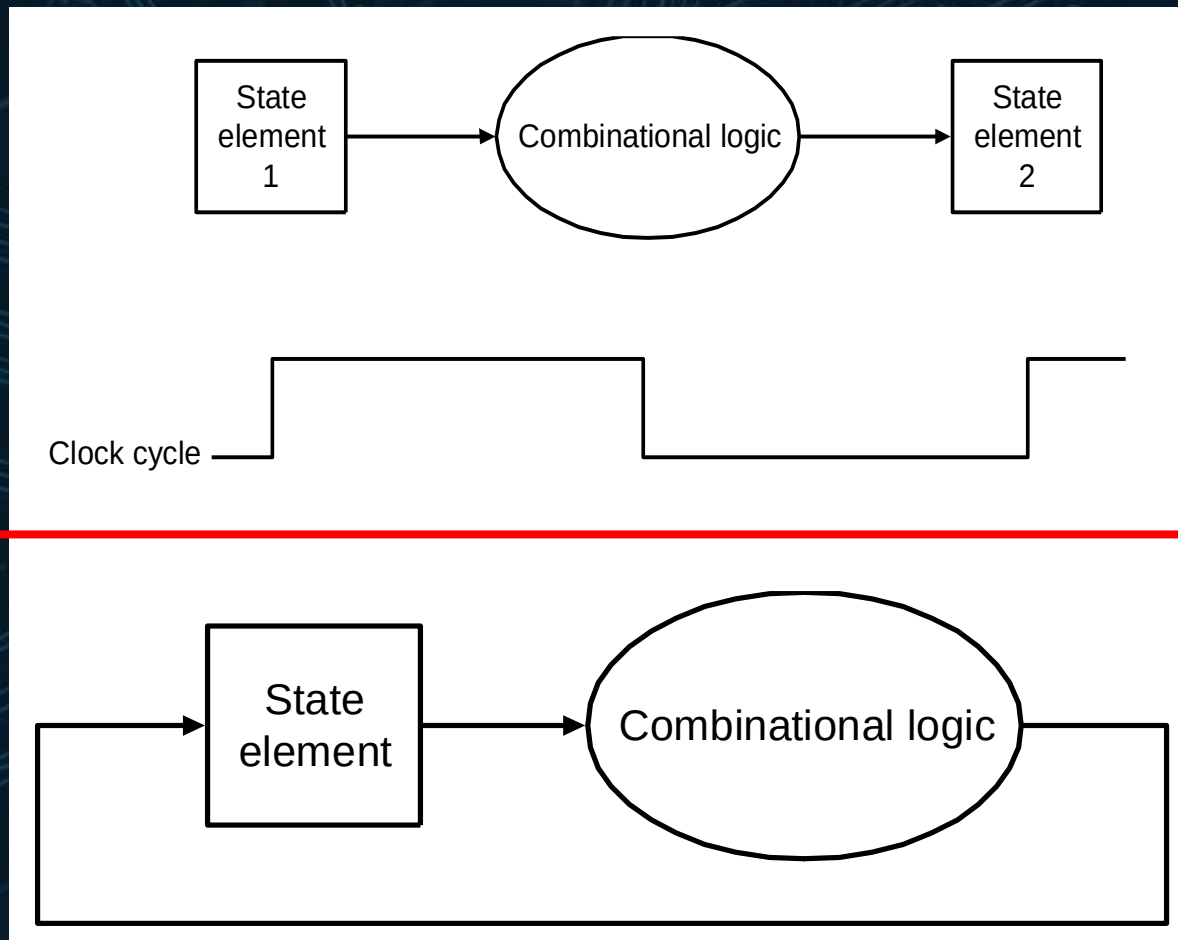
HBM演化



資料源：業界規格彙整 (示意)

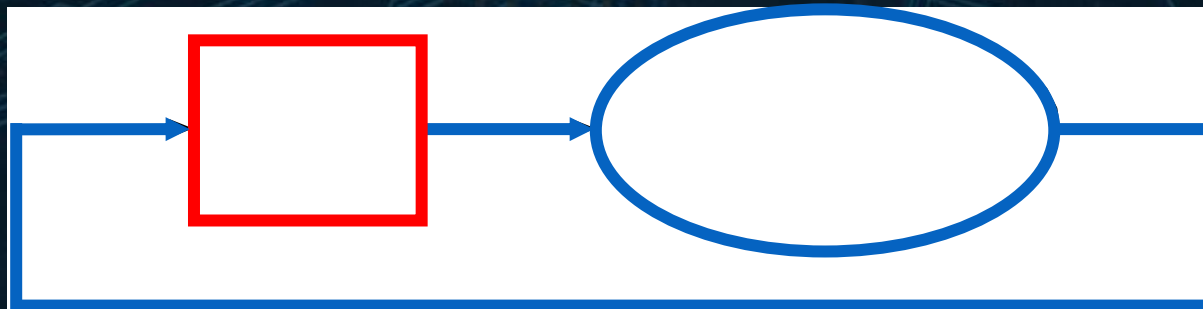
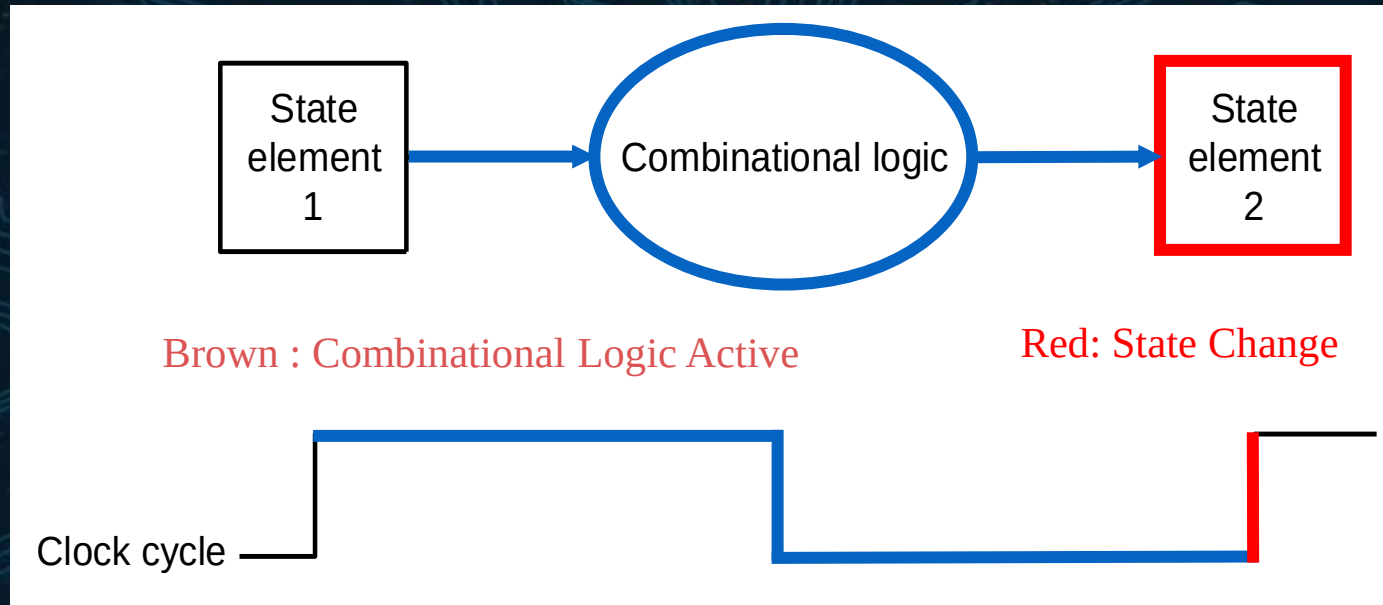


Processor = Combinational Circuits + State Elements



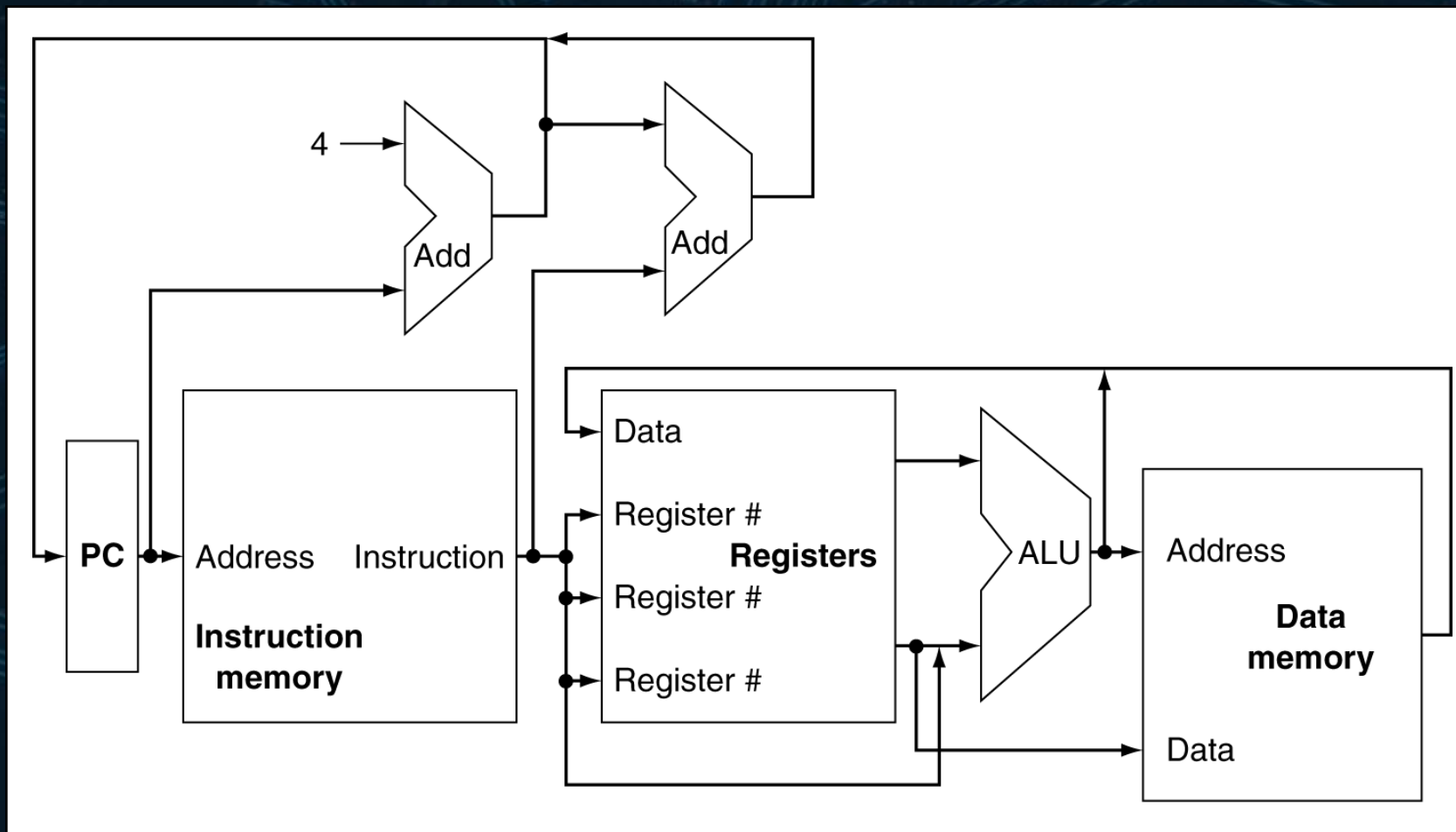


Raising Edge Trigger Example





CPU Overview





一、基本閃鎖器 Basic latch

- 圖3為NOR邏輯閘所構成的記憶體元件。其輸入Set 和Reset 提供電路狀態Q 轉換的工具。
- 比圖3電路更有效的方法如圖4a所示，兩個NOR邏輯閘以交叉耦合 (cross-coupled) 方式連接。此電路為基本閃鎖器 (basic latch)。當 $R = 0$ 且 $S = 1$ ，閃鎖器被設定 (set) 為狀態1。當 $R = 1$ 且 $S = 0$ ，閃鎖器被重置 (reset) 為狀態0。



有NOR邏輯閘的記憶體元件

- 輸入Set 和Reset 提供電路狀態Q 轉換

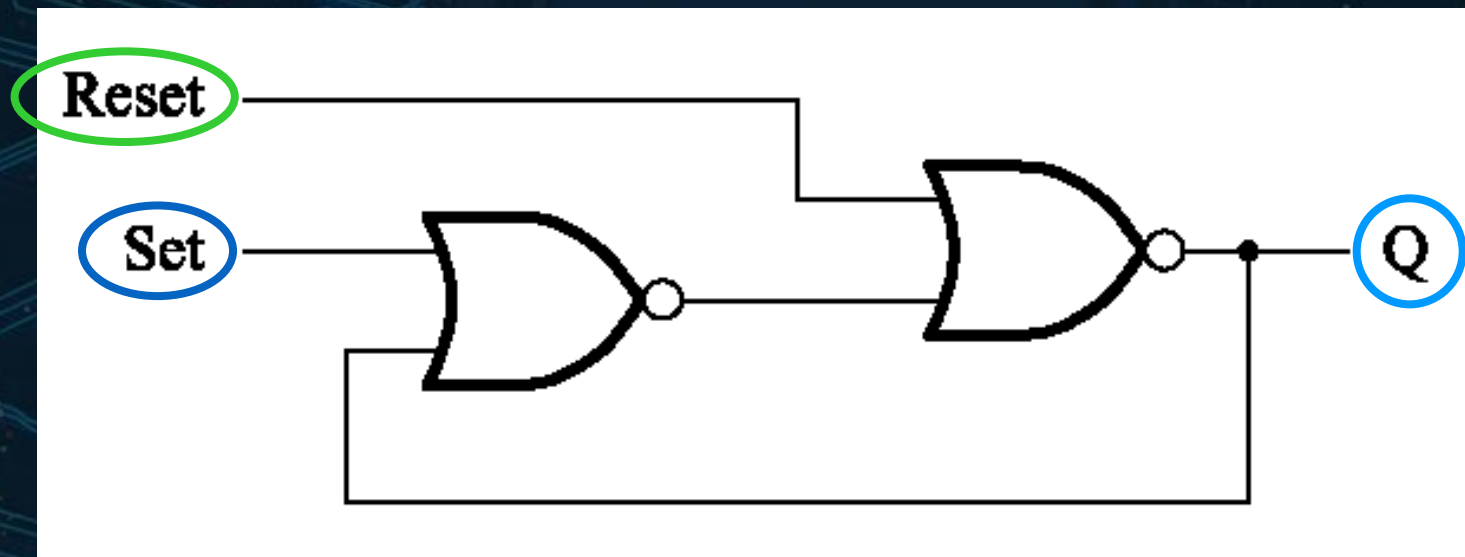
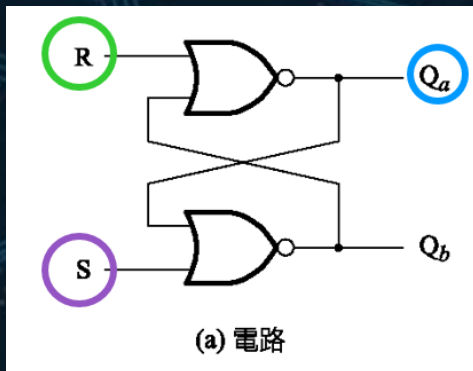


圖3 有NOR邏輯閘的記憶體元件。

NOR邏輯閘的組成的基本閃鎖器

- 兩個NOR邏輯閘以交叉耦合 (cross-coupled) 方式連接



S	R	Q _a	Q _b	
0	0	0/1	1/0	(不變)
0	1	0	1	
1	0	1	0	
1	1	0	0	

(b) 真值表

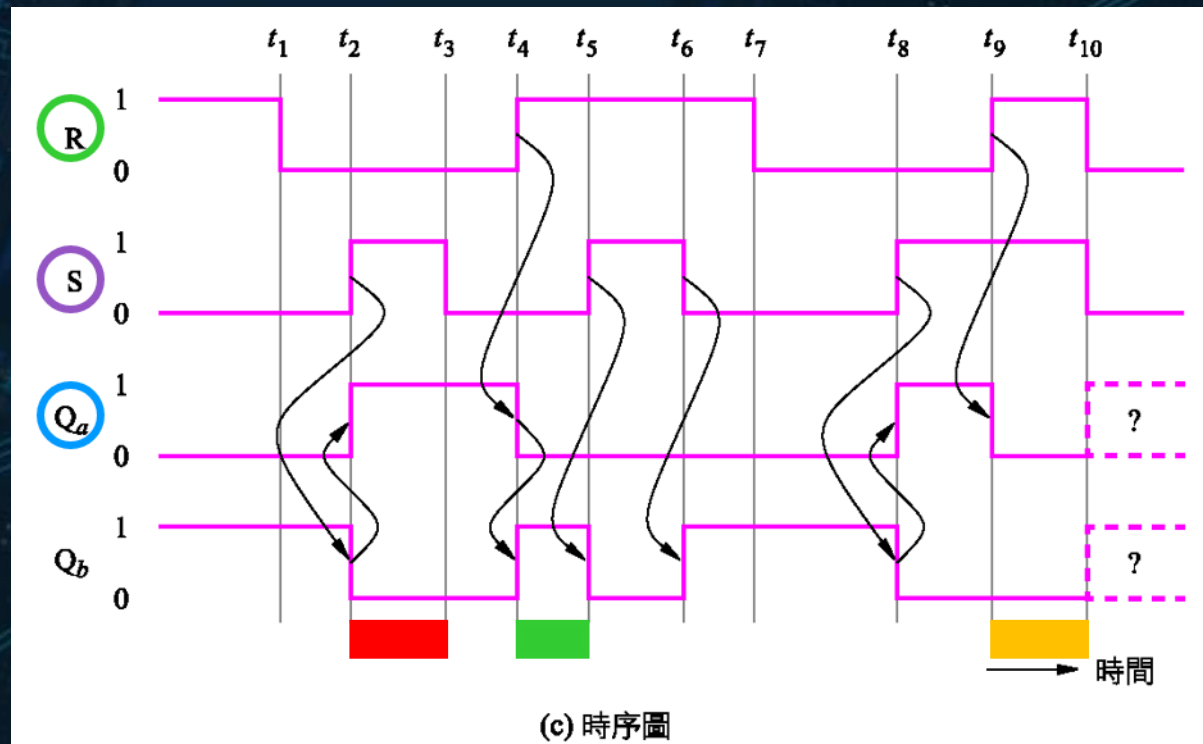
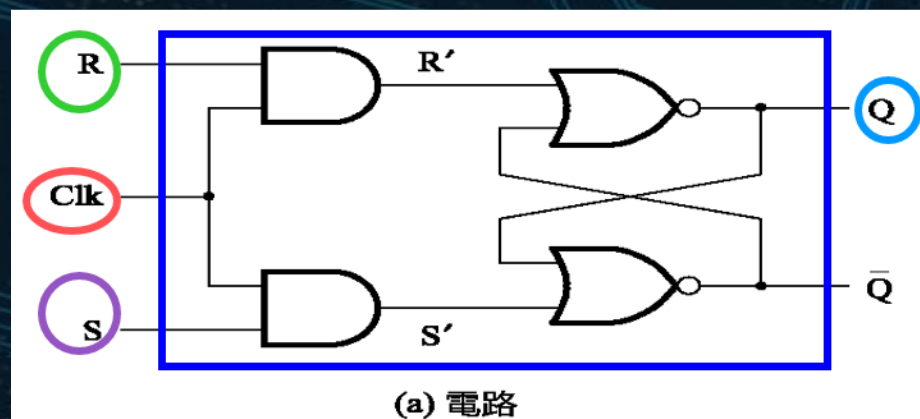


圖4 由NOR邏輯閘的組成的基本閃鎖器。



二、 閘控SR閃鎖器 + Clk 時脈 Gated SR latch

- 定義這些時間間隔的控制訊號通常稱為時脈 (clock) 訊號。名稱 Clk 是反應此訊號的本質。
- 這類使用控制訊號的電路稱為閘控閃鎖器 (gated latch)。因為此電路有設定S和重置R的能力，所以稱為閘控SR閃鎖器 (gated SR latch)。
- 當 $Q = 1$ 時通常稱閃鎖器為設定 (set)，當 $Q = 0$ 時為重置 (reset)。



Clk	S	R	$Q(t+1)$
0	x	x	$Q(t)$ (不變)
1	0	0	$Q(t)$ (不變)
1	0	1	0
1	1	0	1
1	1	1	x

(b) 真值表

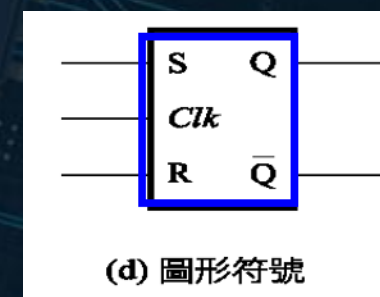
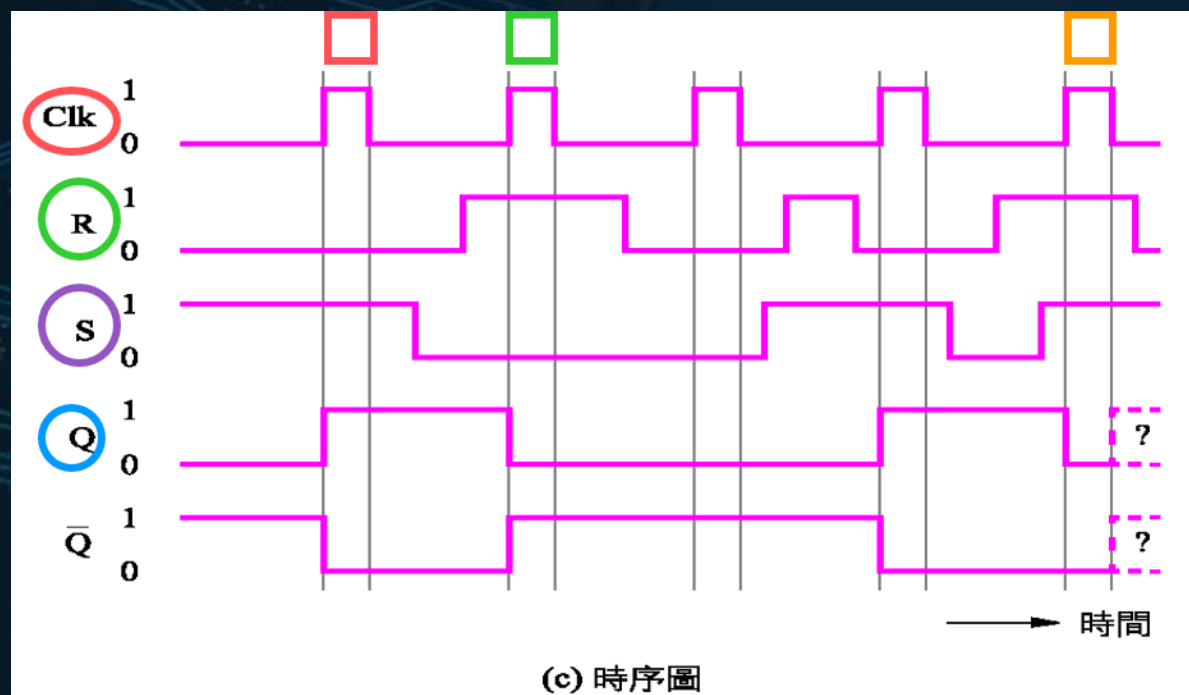


圖5 閘控SR門鎖器。



三、以NAND邏輯閘建構閘控SR閃鎖器 Gated SR latch with NAND gates + NAND

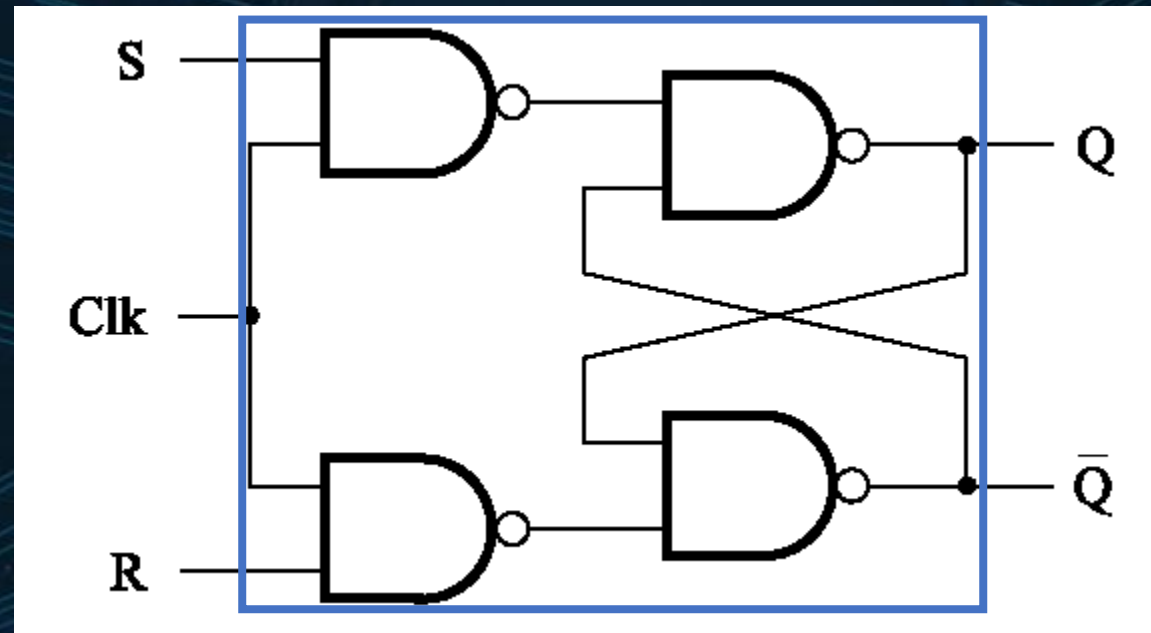


圖6 以NAND邏輯閘建構閘控SR閃鎖器。

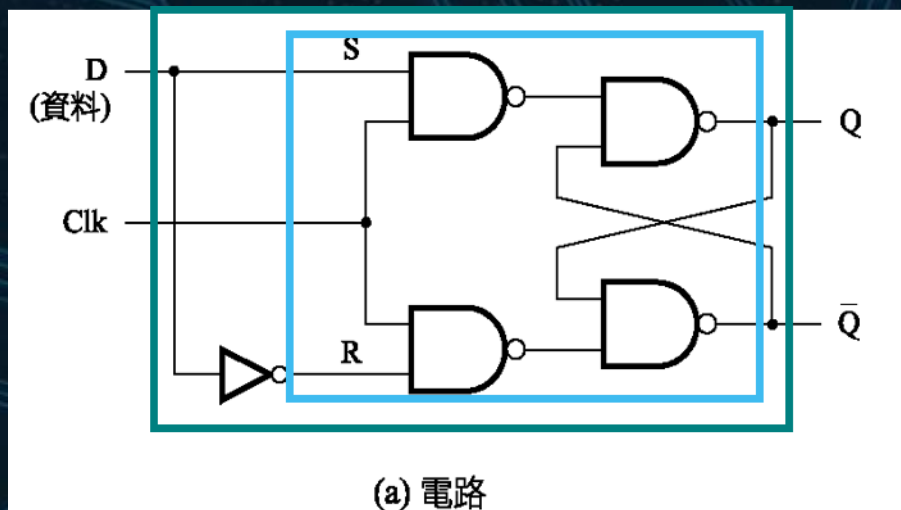


四、 閘控D閃鎖器 + 準位敏感 Gated D latch + level sensitive

- 在時脈訊號的控制下儲存輸入的值，稱為**閘控D閃鎖器 (gated D latch)**。
- 我們可能會想用基本閃鎖器來記憶這些位元，每個閃鎖器一位元。與其說閃鎖器記憶位元值，不如說閃鎖器儲存(store) 位元值較為明確。
- 因為閘控D閃鎖器的輸出是由時脈輸入的準位所控制，此閃鎖器稱為**準位敏感 (level sensitive)**。
- 可以設計一種記憶體元件，其輸出只有在時脈從一個值變成另一個值的時候才會改變。這種電路稱為**邊緣觸發 (edge trigger)**。

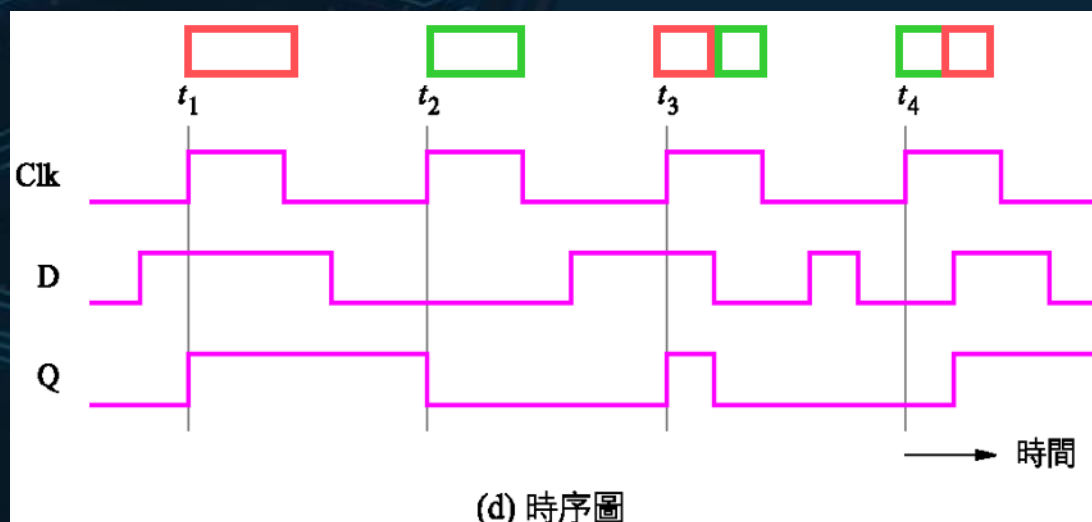
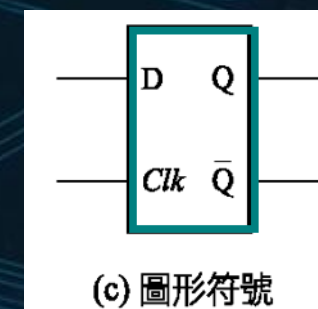


閘控D閃鎖器



(b) 真值表

Clk	D	$Q(t+1)$
0	x	$Q(t)$
1	0	0
1	1	1



準位敏感 (level sensitive)。

Disadv. 傳遞延遲的效應

圖7 閘控D閃鎖器。



五、傳遞延遲的效應 Effects of propagation delays

- 圖8說明此關鍵時間區域。
- 在Clk訊號負緣之前 D 訊號必須為穩定的最小時間，稱為閃鎖器的設置時間 (setup time) t_{su} 。
- 在Clk訊號負緣之後 D 訊號必須保持穩定的最小時間，稱為閃鎖器的保持時間 (hold time) t_h 。

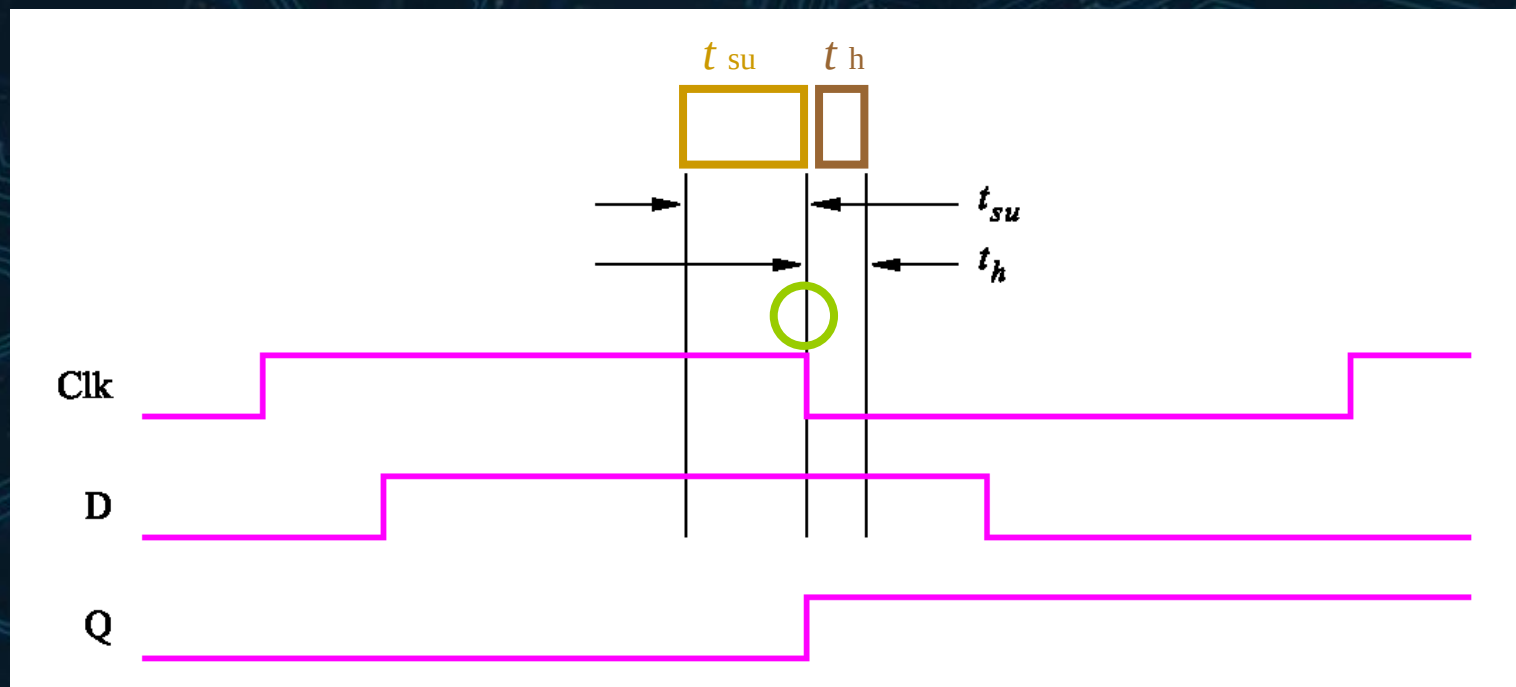
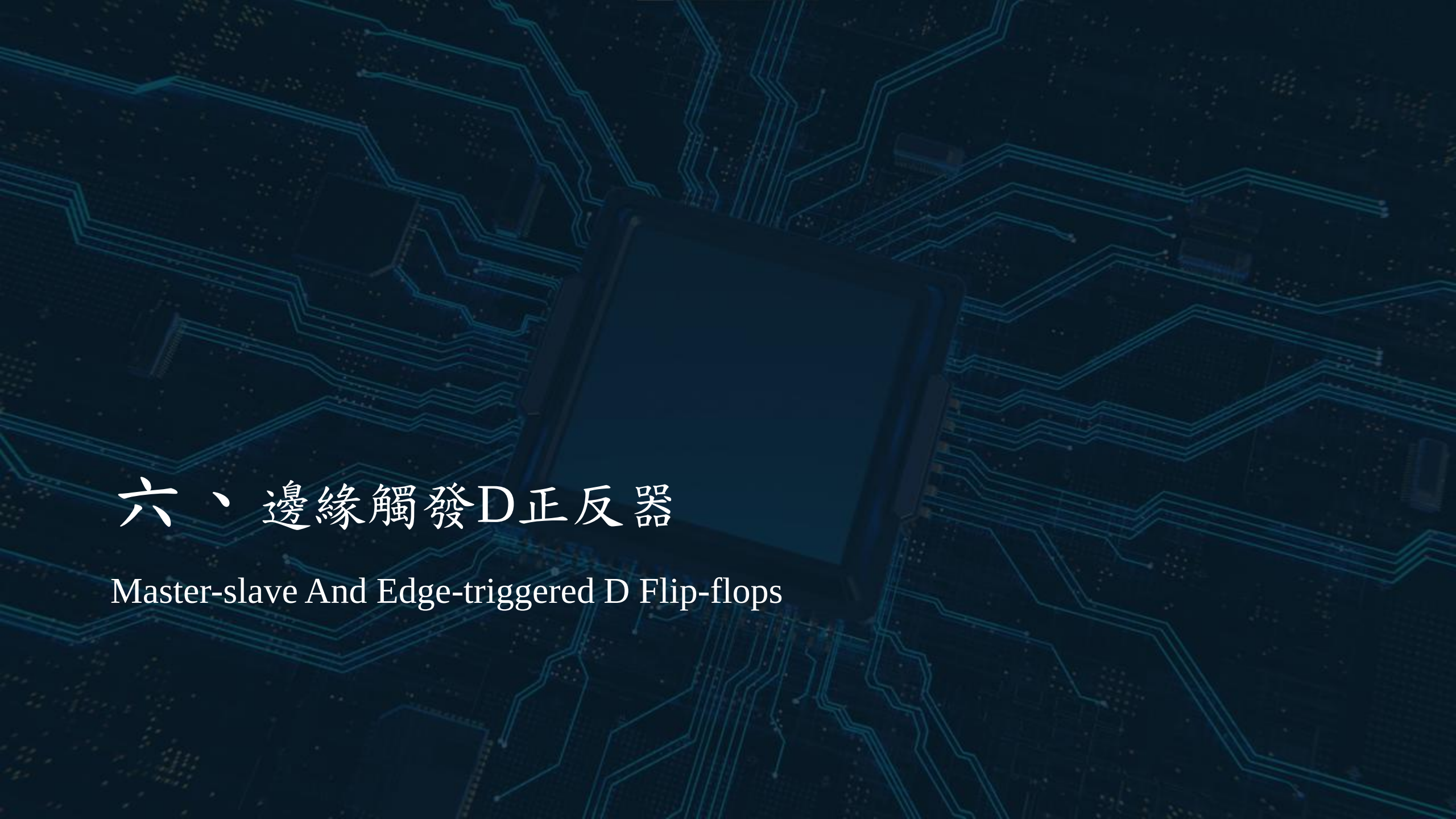


圖8 設定和保持時間。



六、邊緣觸發D正反器

Master-slave And Edge-triggered D Flip-flops



八、正緣觸發D正反器

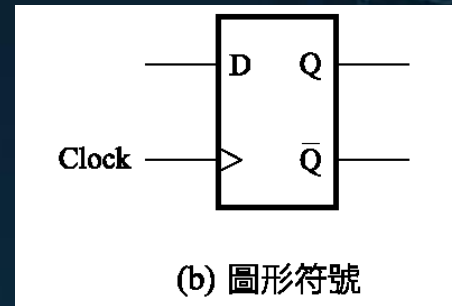
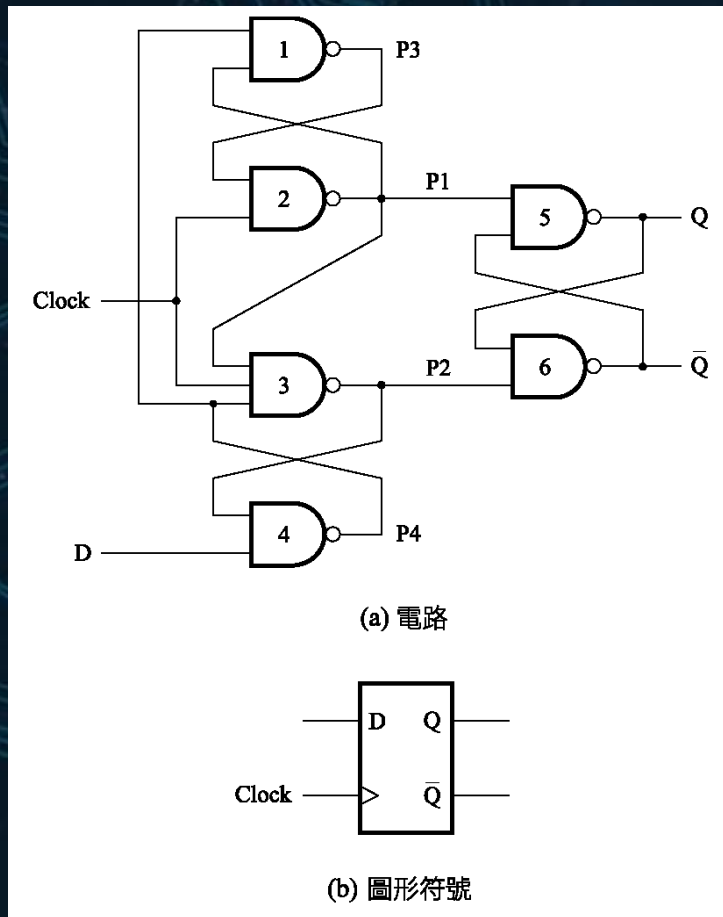


圖10 正緣觸發D正反器。



準位敏感 v.s. 邊緣觸發儲存元件

Level-sensitive Versus v.s. Edge-triggered Storage Elements

閘控D門鎖器 (準位敏感)



正緣觸發D正反器 (正緣觸發)



主從D正反器 (負緣觸發)

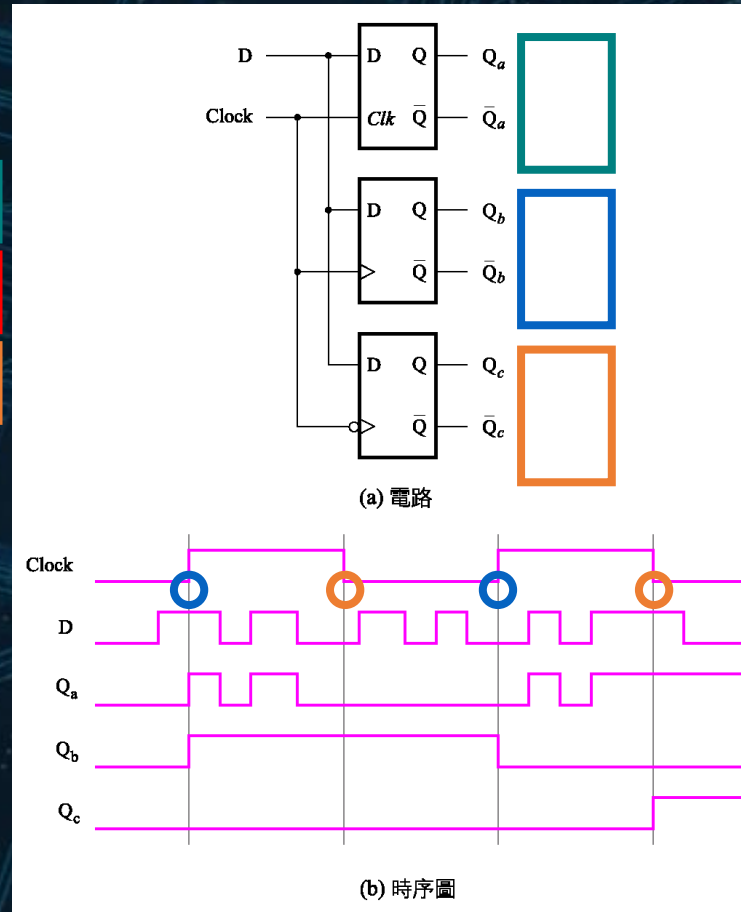


圖11 準位敏感和邊緣觸發D儲存元件的比較。



九、具有清除和預置的D正反器 D Flip-flops With Clear And Preset

- 在這類電路中，clear訊號是用來清除正反器，不考慮時脈訊號，稱此正反器具有非同步清除(asynchronous clear)。
- 實際上，我們通常偏好在時脈的有效邊緣清除正反器，這種同步清除(synchronous clear)如圖12所示。



正緣觸發D正反器

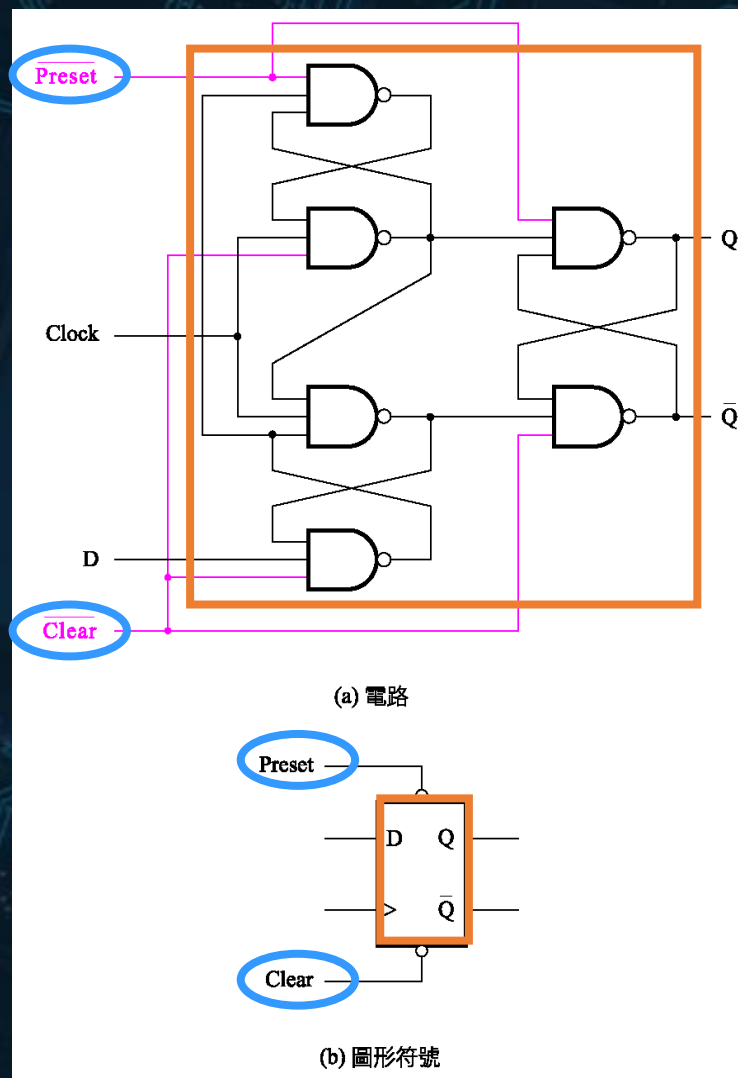


圖13 具有Clear和Preset的正緣觸發D正反器。



D正反器 的同步重置

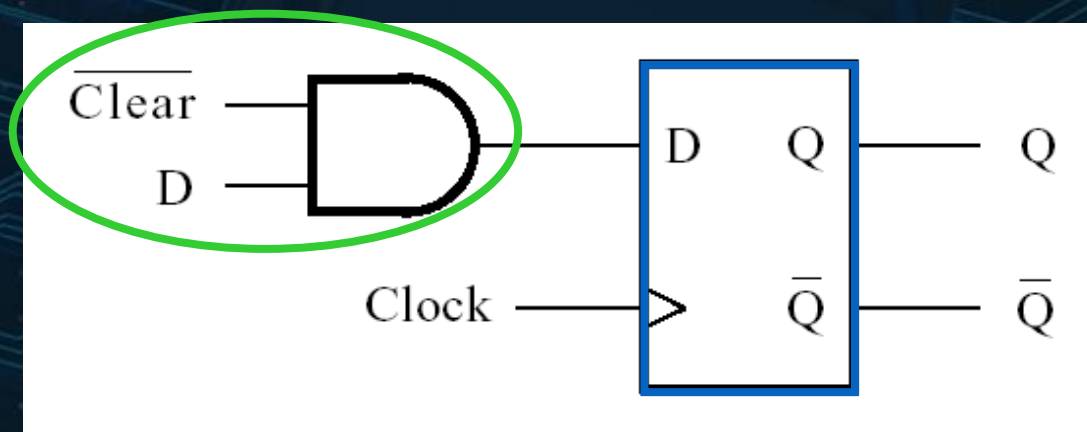
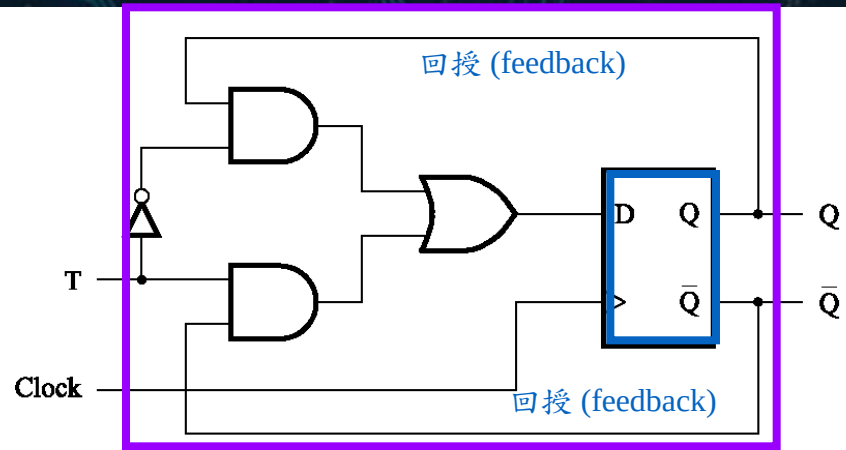


圖14 D正反器 的同步重置 (synchronous Preset) 。



十、T 正反器 T Flip-flop

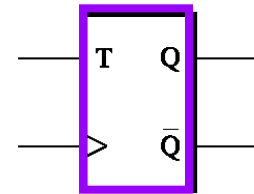
- 回授 (feedback) 連接使得在訊號T 的控制下，輸入訊號D 等於Q或 $\bar{Q}$ 的值。
- 此電路運作的真值表如圖15b所示。實作此真值表的任何電路稱為**T正反器 (T flip-flop)**。



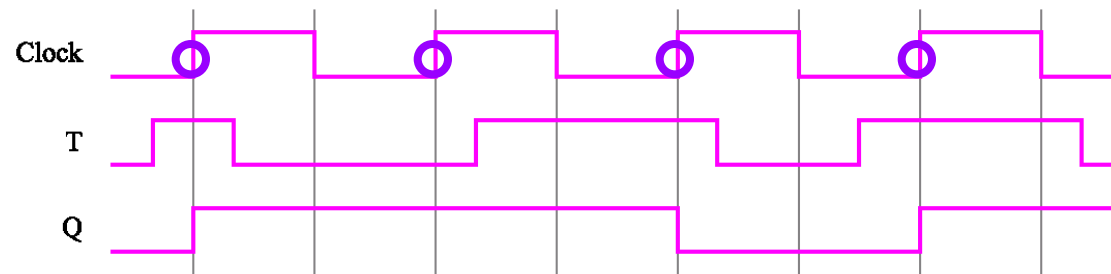
(a) 電路

T	$Q(t+1)$
0	$Q(t)$
1	$\bar{Q}(t)$

(b) 真值表



(c) 圖形符號



(d) 時序圖

圖15 T正反器 回授 (feedback)。



十一、可設定的正反器 (D、T) Configurable Flip-flops

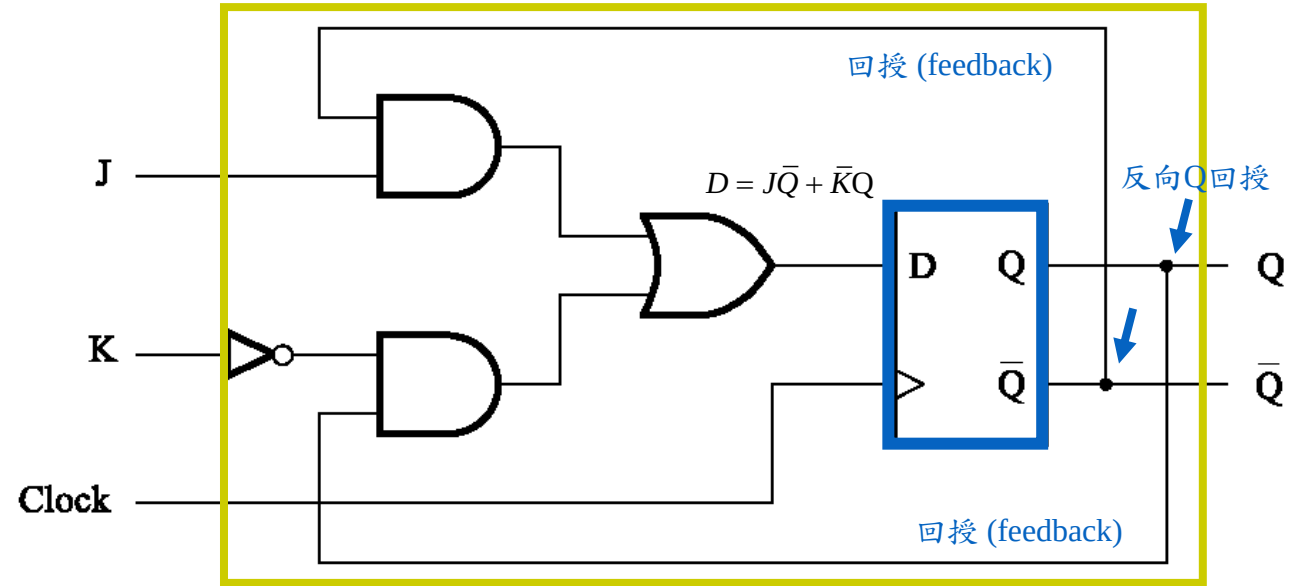
- 一般用途晶片例如PLD提供的正反器有時候是**可設定的 (configurable)**，正反器電路可以設定成 D、T 或其他類型。



十二、JK正反器 JK Flop-flop

反向Q回授

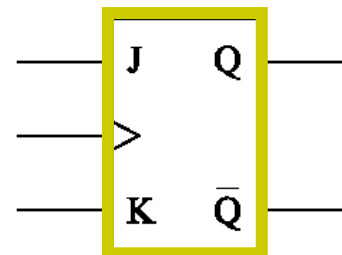
- 對此電路而言，輸入 D 定義為 $D = J\bar{Q} + \bar{K}Q$
- 對應的真值表如圖16b所示。此電路稱為JK正反器 (JK flip-flop)。



(a) 電路

J	K	$Q(t+1)$
0	0	$Q(t)$
0	1	0
1	0	1
1	1	$\bar{Q}(t)$

(b) 真值表



(c) 圖形符號

圖16 JK正反器。



十三、術語總結 Summary Of Terminology

- 我們使用的術語是很常用的。但是必須注意閂鎖器 (latch) 和正反器 (flip-flop) 有不同的解釋。我們的術語總結如下：
 - 基本閂鎖器 (basic latch) 為兩個NOR邏輯閘或兩個NAND邏輯閘的回授連接，可以儲存一位元資訊。它可以用S輸入設定為1，用R輸入重置為0。
 - 閂控閂鎖器 (gated latch) 為包含輸入閂控和控制輸入訊號的基本閂鎖器。



- 兩種閘控閂鎖器：
 - 閘控SR閂鎖器 (gated SR latch) 使用S 和R 輸入分別將閂鎖器設定為1或重置為0。
 - 閘控D閂鎖器 (gated D latch)使用D 輸入迫使閂鎖器狀態值和D 輸入相同。
- 正反器 (flip-flop) 是以閂鎖器為基礎的儲存元件，其輸出狀態只在控制時脈訊號的邊緣改變。我們考慮兩種正反器：
 - 邊緣觸發的正反器 (edge-triggered flip-flop) 只有當時脈有效邊緣發生時的輸入值影響。



十四、暫存器 Registers

共通時脈的n位元D正反器

- 一個正反器儲存一位元的資訊。當一組n個正反器儲存n位元資訊例如n位元數值，我們稱這些正反器為暫存器 (register)。
- 暫存器中的每個正反器使用共通的時脈，且每個正反器運作如同前一節所述。
- 暫存器一詞只是方便稱呼由正反器組成的n位元架構。



十五、位移暫存器 Shift Register

- 可以位移內容的暫存器稱為**位移暫存器 (shift register)**。

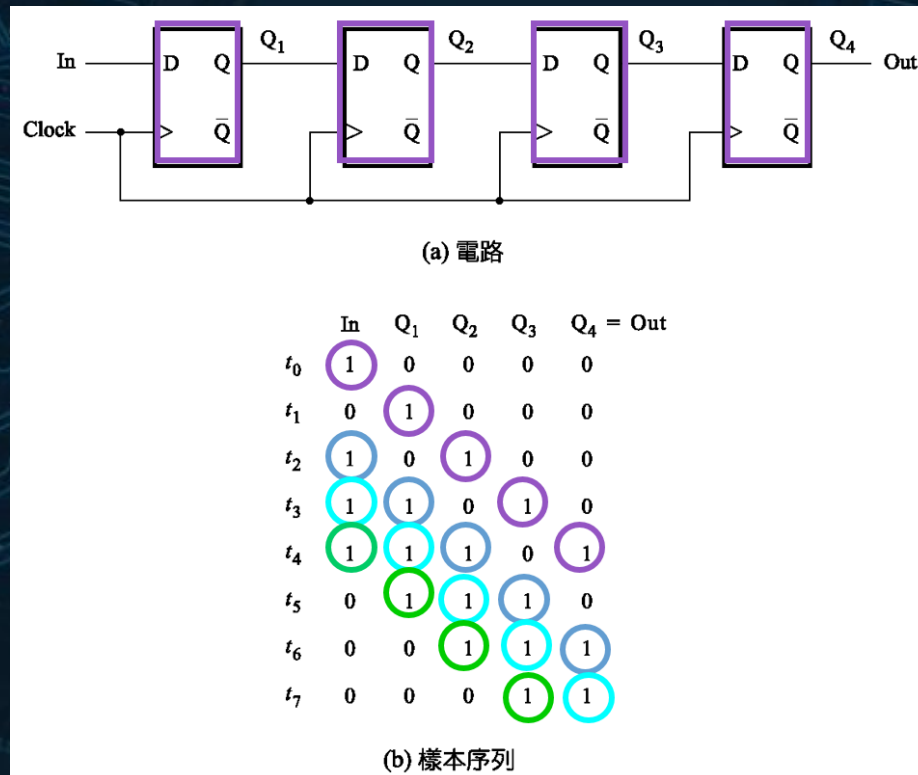
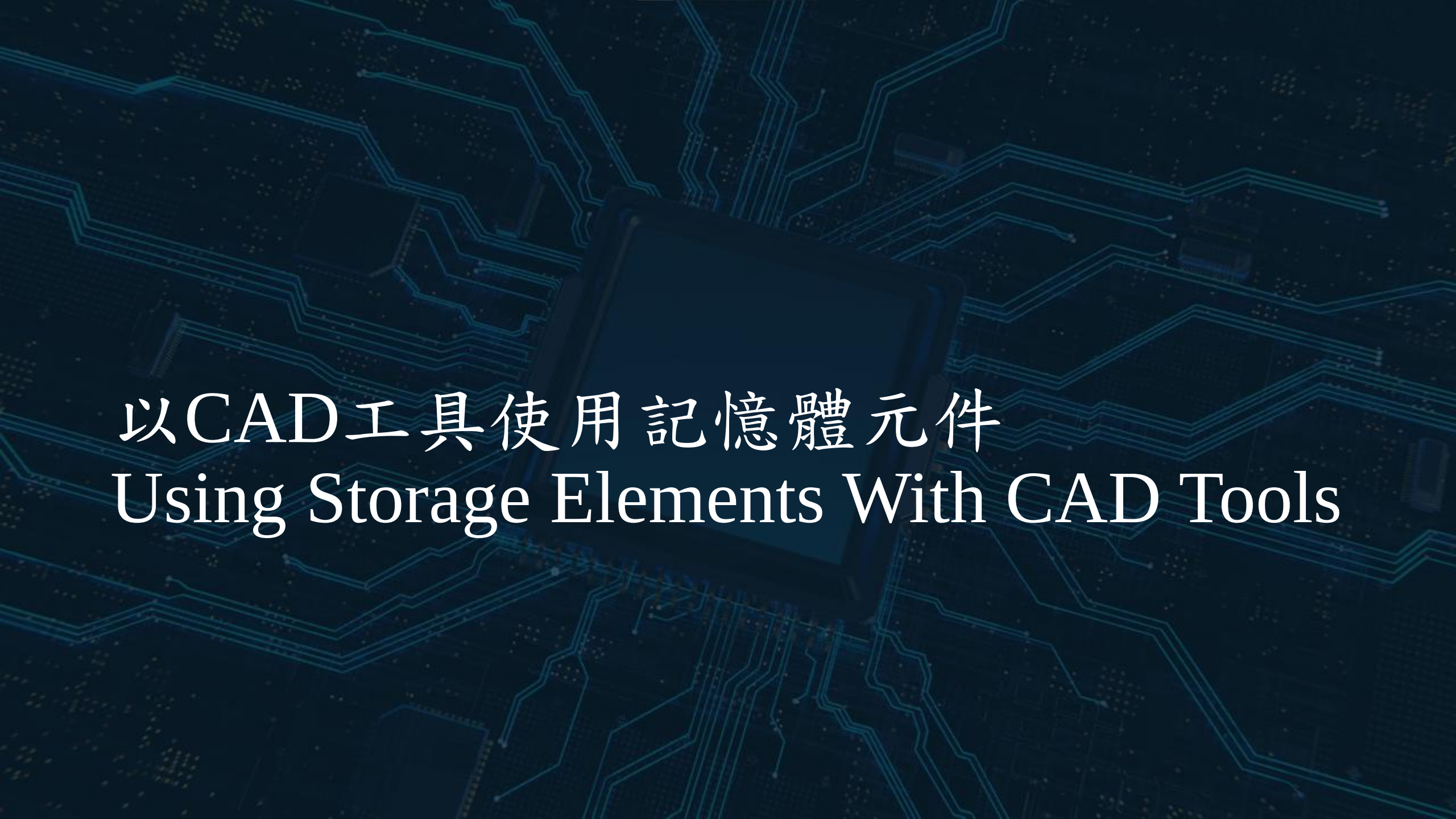


圖17 簡單位移暫存器。



十六、平行存取位移暫存器 Parallel-access Shift Register

- 在電腦系統中常常需要傳送 n 位元資料項。可以用 n 條分開的線同時傳輸所有位元，稱為**平行(parallel)**傳送。
- 但是也可以單一條線傳輸所有位元，一次一個位元，連續 n 個時脈循環。此為**串列(serial)**傳送。



以CAD工具使用記憶體元件

Using Storage Elements With CAD Tools



於圖示輸入中包含記憶體元件 Including Storage Elements In Schematics

- 此時序圖說明晶片輸入接腳的時脈正緣發生，到正反器輸出的改變出現在晶片輸出接腳的時間延遲。這時間稱為時脈至輸出時間 (clock-to-output time) t_{co} 。

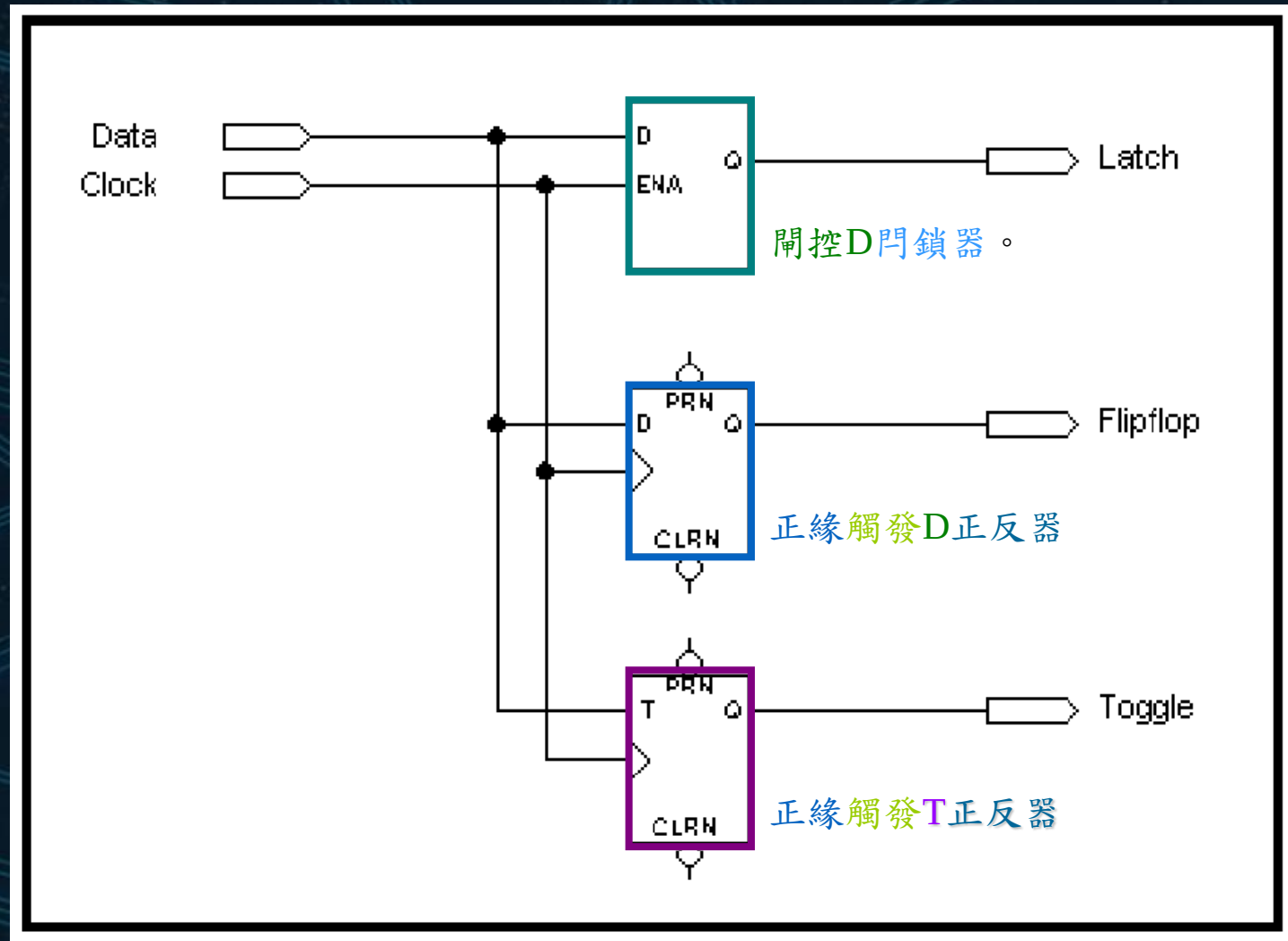


圖30 圖示中的三種儲存元件。



CAD工具產生的閘控D閃鎖器

- 但是若沒有此AND邏輯閘，電路會產生時序問題，稱為靜態危機 (static hazard)。

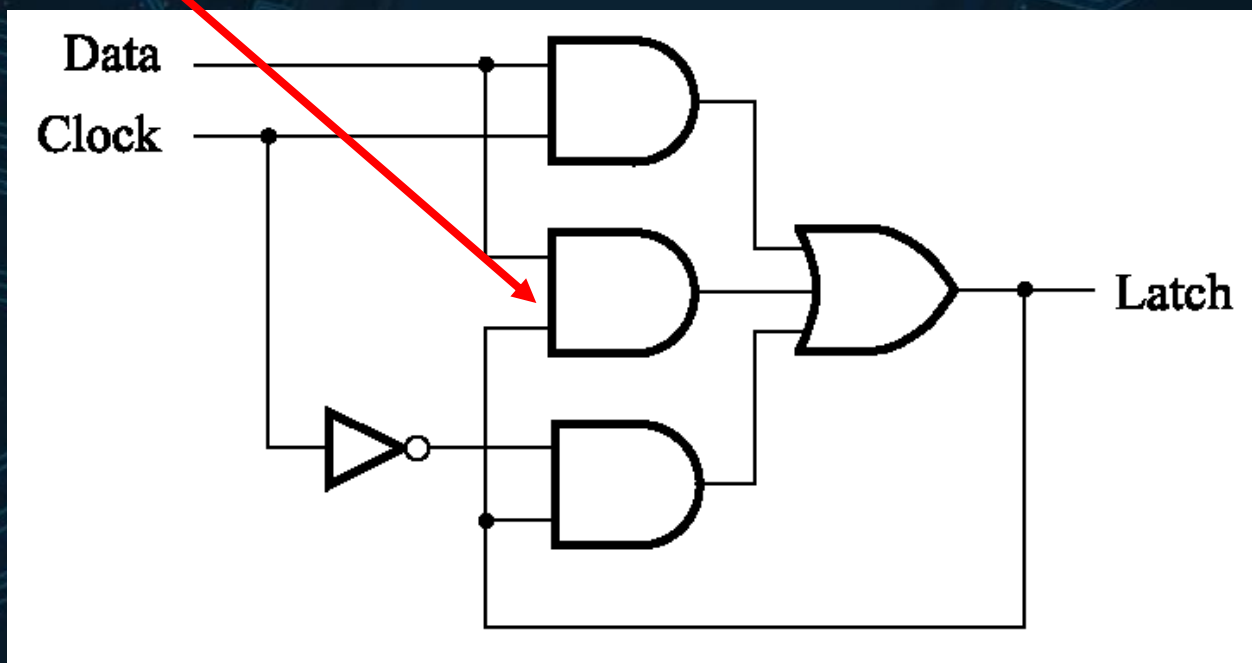


圖31 CAD工具產生的閘控D閃鎖器。

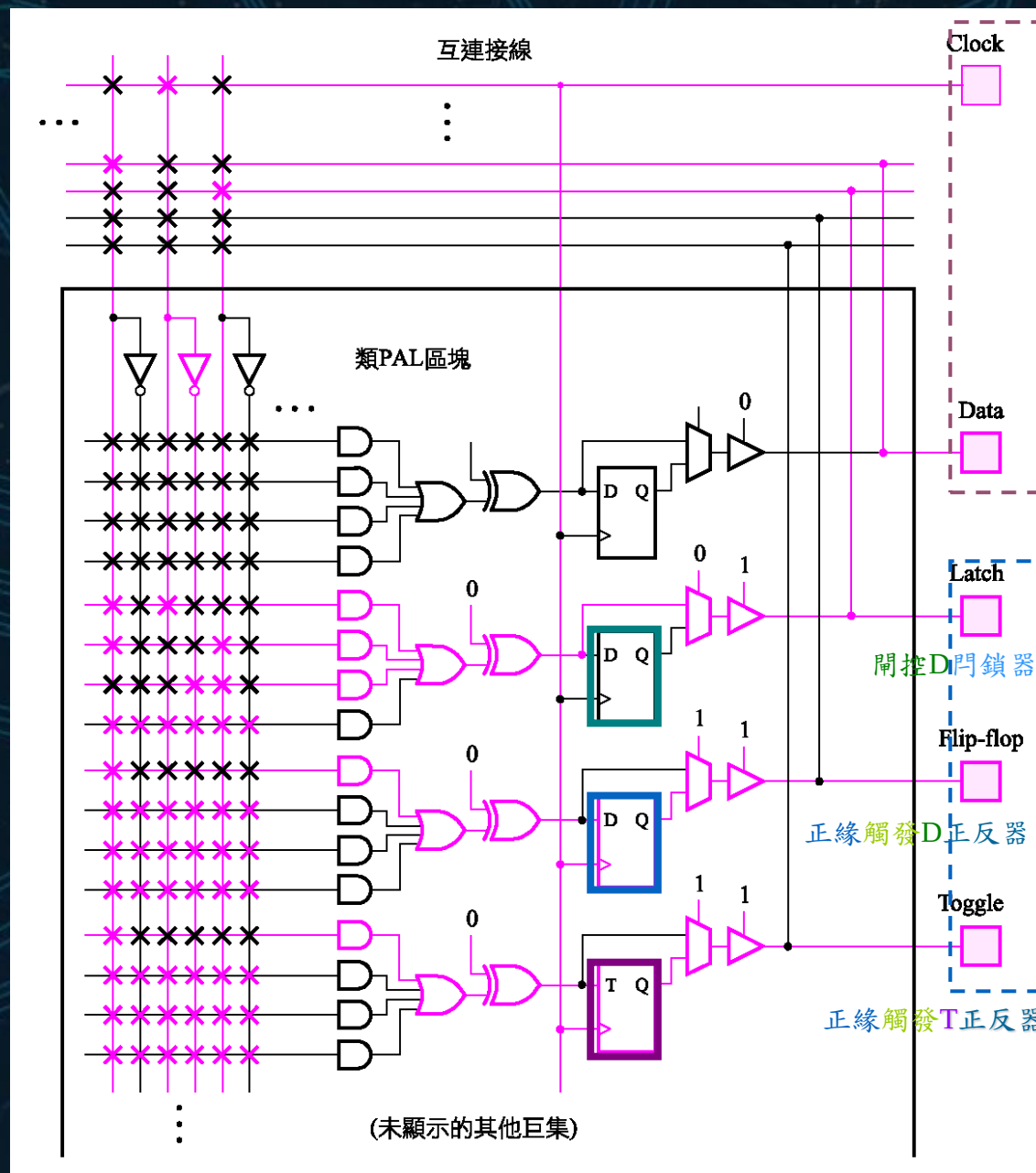


圖32 圖30 的圖示實作於CPLD。

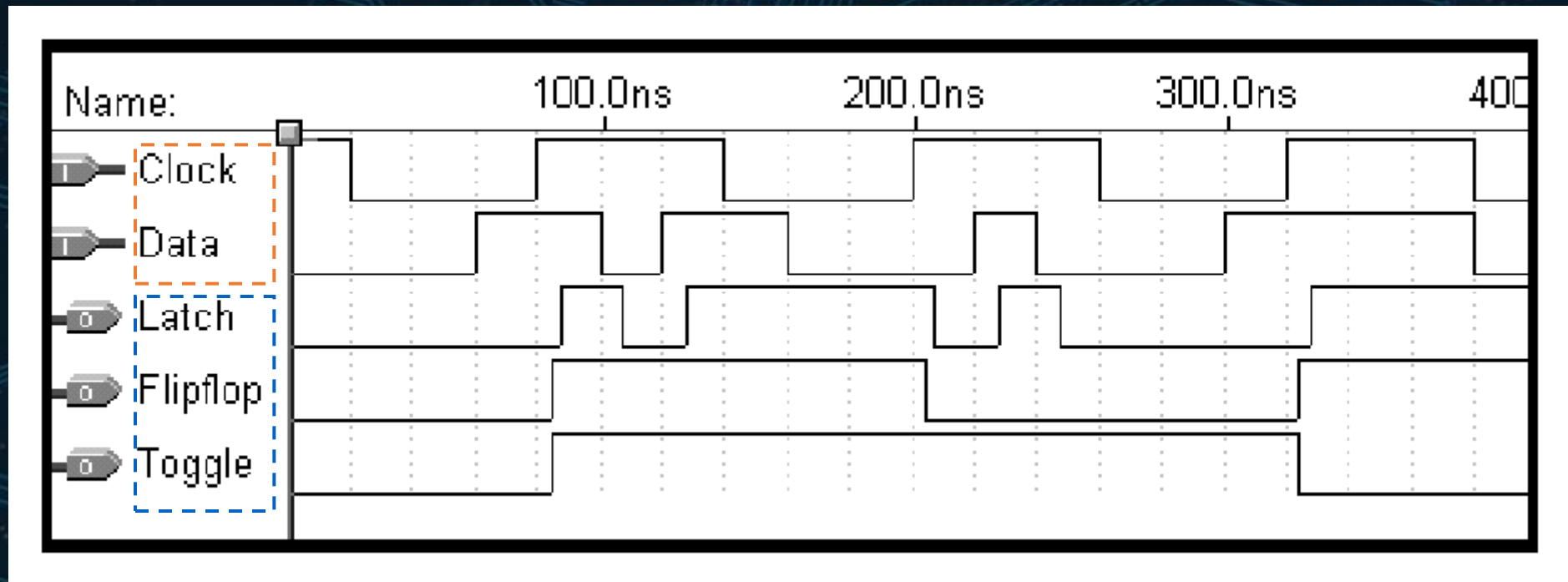


圖33 圖32儲存元件的時序模擬。



對儲存元件使用VHDL順序敘述

- 重複圖36於圖35，為具有隱含記憶體的VHDL程式碼範例。因為程式碼沒有指定IF敘述的條件,不成立時AeqB訊號值應該為何?語義指定此時AeqB應該保持目前的值。隱含的記憶體是**描述循序電路(Sequential Circuit)元件**的關鍵概念。



```
LIBRARY ieee ;  
USE ieee.std_logic_1164.all ;  
  
ENTITY implied IS  
    PORT ( A, B      : IN      STD_LOGIC ;  
           AeqB      : OUT    STD_LOGIC ) ;  
END implied ;  
  
ARCHITECTURE Behavior OF implied IS  
BEGIN  
    PROCESS ( A, B )  
    BEGIN  
        IF A = B THEN  
            AeqB <= '1' ;  
        END IF ;  
    END PROCESS ;  
END Behavior ;
```

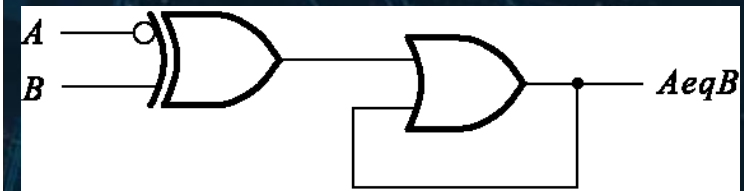


圖 35 圖36 的程式碼，說明隱含的記憶體



範例一

- 閘控D閃鎖器的程式碼 圖37的程式碼定義實體latch，其輸入為D和Clk，輸出為Q。
- 此程序用一個if-then-else敘述來定義Q輸出的值。
 - 當Clk = 1，Q取D的值。
 - 當Clk不是1，程式碼沒有指定Q值應該為何。因此Q會保持其目前的值，此程式碼即為閘控D閃鎖器。
- 程序的敏感列表包含D和Clk，因為這些訊號會使得Q輸出的值改變。



```
LIBRARY ieee ;  
USE ieee.std_logic_1164.all ;  
  
ENTITY latch IS  
    PORT ( D, Clk  : IN    STD_LOGIC ;  
          Q       : OUT   STD_LOGIC) ;  
END latch ;  
  
ARCHITECTURE Behavior OF latch IS  
BEGIN  
    PROCESS ( D, Clk )  
    BEGIN  
        IF Clk = '1' THEN  
            Q <= D ;  
        END IF ;  
    END PROCESS ;  
END Behavior ;
```

圖37 閘控D閃鎖器的程式碼。



範例二

- D正反器的程式碼 圖38定義模組flipflop，為正緣觸發D正反器。此程式碼和圖37大致相同，只有兩個地方不一樣。
 - 第一個，程序敏感列表只包含時脈訊號，因為它是唯一可以使得Q輸出改變的訊號。
 - 第二個，if-then-else敘述用的條件和閂鎖器不同。
- 語法Clock'EVENT使用VHDL的**特性(attribute)**。對應到物件（例如訊號）的性質。此例中' EVENT特性表示Clock訊號的改變。
- Clock'EVENT條件和Clock = 1條件合併起來表示『Clock訊號值剛改變，且其值現在為1』。
- 這個條件相當於時脈正緣。因為Q輸出只在時脈正緣改變，故此程式碼表示正緣觸發D正反器。



```
LIBRARY ieee ;  
USE ieee.std_logic_1164.all ;  
  
ENTITY flipflop IS  
    PORT ( D, Clock      : IN  STD_LOGIC ;  
          Q              : OUT STD_LOGIC) ;  
END flipflop ;  
  
ARCHITECTURE Behavior OF flipflop IS  
BEGIN  
    PROCESS ( Clock )  
    BEGIN  
        IF Clock'EVENT AND Clock = '1' THEN  
            Q <= D ;  
        END IF ;  
    END PROCESS ;  
END Behavior ;
```

正緣觸發

圖 38 D正反器的程式碼。



範例三

- D正反器的另一種程式碼，圖39的程序使用和圖38不同的方法來描述D正反器。它使用敘述**WAIT UNTIL Clock'EVENT AND Clock = '1'**。
- 此敘述的效果和if敘述一樣。使用WAIT UNTIL敘述的程序為特殊情況，因為敏感列表被省略了。
- **WAIT UNTIL**架構表示敏感列表只包含時脈訊號。使用VHDL作為電路合成時，唯有它是程序中的第一個敘述，才可以使用WAIT UNTIL敘述。
- 一般來說，VHDL的時脈正緣觸發正反器都是使用條件Clock'EVENT AND Clock = '1'。
- 當此條件出現在IF敘述，任何IF敘述內部被指定值的訊號都會實作成正反器輸出。若使用此條件於WAIT UNTIL敘述，整個程序中被指定值的訊號都會實作成正反器輸出。



```
LIBRARY ieee;
USE ieee.std_logic_1164.all;

ENTITY flipflop IS
    PORT ( D, Clock : IN      STD_LOGIC ;
          Q          : OUT STD_LOGIC);
END flipflop ;

ARCHITECTURE Behavior OF flipflop IS
BEGIN
    PROCESS
    BEGIN
        WAIT UNTIL Clock'EVENT AND Clock = '1';
        Q <= D ;
    END PROCESS ;
END Behavior ;
```

圖 39 和圖38 等效的程式碼，使用 WAIT UNTIL敘述



範例四 非同步重置D正反器

- 圖40的程序類似於圖38的程序，描述具有非同步低值有效重置（清除）輸入的D正反器。當重置輸入Resetn為0時，正反器的Q輸出被設成0。



```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY flipflop IS
    PORT ( D, Resetn, Clock: IN          STD_LOGIC ;
          Q          : OUT STD_LOGIC) ;
END flipflop ;

ARCHITECTURE Behavior OF flipflop IS
BEGIN
    PROCESS ( Resetn, Clock )
    BEGIN
        IF Resetn = '0' THEN
            Q <= '0' ;
        ELSIF Clock'EVENT AND Clock = '1' THEN
            Q <= D ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖 40 具有非同步重置的D正反器



範例五 同步重置D正反器

- 圖41說明如何描述具有同步重置輸入的D正反器。此例的重置訊號只有當時脈正緣到達時才會動作。此程式碼產生圖14的電路，有個**AND邏輯閘**接到**正反器的D輸入**。



```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY flipflop IS
    PORT ( D, Resetn, Clock: IN      STD_LOGIC ;
          Q      : OUT STD_LOGIC) ;
END flipflop ;

ARCHITECTURE Behavior OF flipflop IS
BEGIN
    PROCESS
    BEGIN
        WAIT UNTIL Clock'EVENT AND Clock = '1' ;
        IF Resetn = '0' THEN
            Q <= '0' ;
        ELSE
            Q <= D ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖 41 具有同步重置D正反器



使用VHDL順序敘述作為暫存器和計數器

Using VHDL Constructs for Registers and Counters

- 在VHDL中，我們可以用**順序敘述**來描述電路，不必實體化預先定義好的暫存器、位移暫存器、計數器等等子電路。
- 圖40為D正反器的程式碼。直觀描述n位元暫存器的方式是撰寫**階層性程式碼**，包含n個D正反器子電路實體。
- 比較簡單的方法如圖46所示，其程式碼和圖40相同，只是D輸入和Q輸出被定義為多位元訊號。此程式碼代表具有非同步清除的八位元暫存器。



非同步清除的八位元暫存器

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY reg8 IS
    PORT (
        D          : IN      STD_LOGIC_VECTOR(7 DOWNTO 0);
        Resetn, Clock : IN      STD_LOGIC ;
        Q          : OUT      STD_LOGIC_VECTOR(7 DOWNTO 0) );
END reg8 ;

ARCHITECTURE Behavior OF reg8 IS
BEGIN
    PROCESS ( Resetn, Clock )
    BEGIN
        IF Resetn = '0' THEN
            Q <= "00000000" ;
        ELSIF Clock'EVENT AND Clock = '1' THEN
            Q <= D ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖 46 具有非同步清除的八位元暫存器的程式碼



範例六 N位元暫存器

- 因為邏輯電路常常需要不同尺寸的暫存器，定義一個正反器個數可以輕易改變的暫存器實體是很方便的。
- 圖47為圖46的程式碼延伸成包含設定正反器個數的參數。參數為整數N，由VHDL架GENERIC所定義。**N值以 := 指定運算子被設定成16**。藉由改變此參數，程式碼可以表示任何尺寸的暫存器。
- 若暫存器被宣告為元件，則可以做為其他程式碼的子電路。該程式碼可以使用GENERIC參數的預設值，或是用**GENERIC MAP**架構指定不同的參數。圖45為說明如何使用GENERIC MAP的例子。
- 圖47的D和Q訊號以N定義。此敘述以語法 $Q \leq (\text{OTHERS} \Rightarrow '0')$ 將Q的所有位元重置為0。對預設值 $N = 16$ ，此敘述等效於敘述 $Q \leq "0000000000000000"$ 。語法 $\text{OTHERS} \Rightarrow '0'$ 對Q的每個位元指定'0'，不管Q有多少位元。任何N值都可以使用此程式碼，而不只是 $N = 16$ 。



```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY regn IS
    GENERIC ( N : INTEGER := 16 ) ;
    PORT ( D          : IN      STD_LOGIC_VECTOR(N-1 DOWNT0 0) ;
          Resetn, Clock : IN      STD_LOGIC ;
          Q            : OUT     STD_LOGIC_VECTOR(N-1 DOWNT0 0) ) ;
END regn ;

ARCHITECTURE Behavior OF regn IS
BEGIN
    PROCESS ( Resetn, Clock )
    BEGIN
        IF Resetn = '0' THEN
            Q <= (OTHERS => '0') ;
        ELSIF Clock'EVENT AND Clock = '1' THEN
            Q <= D ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖 47 具有非同步清除的n位元暫存器的程式碼



範例七 四位元位移暫存器

- 使用四個子電路。每個子電路都由一個D正反器和二對一多工器連接到D輸入所組成。
- 圖48定義實體 *muxdff* 代表此子電路。兩個資料輸入為D0和D1，以Sel輸入來選擇。程序敘述指定若Sel = 0，在時脈正緣將D0值指定給Q；否則，將D1值指定給Q。
- 圖49定義四位元位移暫存器。敘述Stage3實體化最左邊輸出為Q3的正反器，敘述Stage0實體化最右邊輸出為Q0的正反器。當L = 1則從R輸入平行載入，當L = 0則由左向右位移。序列資料從w輸入位移進入最重位元Q3。



平行存取位移暫存器

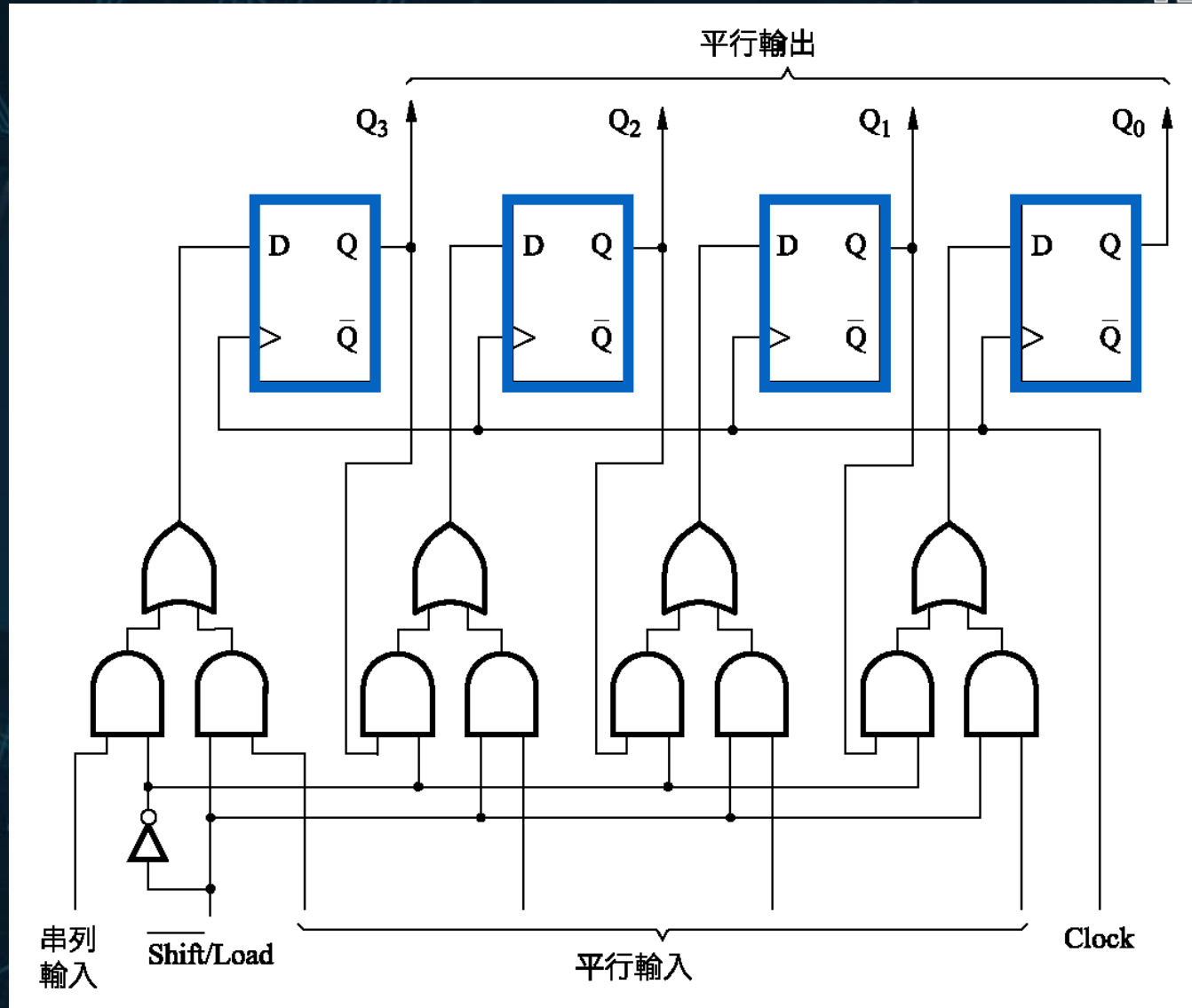


圖18 平行存取位移暫存器。



子電路-二對一多工器

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY muxdff IS
    PORT ( D0, D1, Sel, Clock: IN  STD_LOGIC ;
           Q                               : OUT  STD_LOGIC ) ;
END muxdff ;

ARCHITECTURE Behavior OF muxdff IS
BEGIN
    PROCESS
    BEGIN
        WAIT UNTIL Clock'EVENT AND Clock = '1' ;
        IF Sel = '0' THEN
            Q <= D0 ;
        ELSE
            Q <= D1 ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖 48 具有二對一多工器連接到D輸入的D正反器的程式碼



四位元位移暫存器

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY shift4 IS
    PORT (    R            : IN            STD_LOGIC_VECTOR(3 DOWNT0 0) ;
            L, w, Clock    : IN            STD_LOGIC ;
            Q              : BUFFER        STD_LOGIC_VECTOR(3 DOWNT0 0) ) ;
END shift4 ;

ARCHITECTURE Structure OF shift4 IS
    COMPONENT muxdff
        PORT (    D0, D1, Sel, Clock    : IN            STD_LOGIC ;
                Q              : OUT            STD_LOGIC ) ;
    END COMPONENT ;
BEGIN
    Stage3: muxdff PORT MAP ( w, R(3), L, Clock, Q(3) ) ;
    Stage2: muxdff PORT MAP ( Q(3), R(2), L, Clock, Q(2) ) ;
    Stage1: muxdff PORT MAP ( Q(2), R(1), L, Clock, Q(1) ) ;
    Stage0: muxdff PORT MAP ( Q(1), R(0), L, Clock, Q(0) ) ;
END Structure ;
```

圖 49 四位元位移暫存器階層性程式碼



範例八

- **四位元位移暫存器**的另一種程式碼如圖50所示。每一行程式碼都標上數字以便說明。我們不使用子電路，而是以順序敘述描述位移暫存器。
- 根據第13行的**WAIT UNTIL**敘述，程序內被指定值的任何訊號必會被實作為**正反器的輸出**。
- 第14和15行指定當 $L = 1$ 時位移暫存器的平行載入。第16到20行的**ELSE**敘述指定位移運算。第17行將Q1值位移到輸出為Q0的正反器。第18和19行將Q2和Q3值分別位移到輸出為Q1和Q2的正反器。第20行將w值位移進入最左邊的正反器，其輸出為Q3。根據介紹過的程序語義，第17到20行四個指定被排程，當程序中的所有敘述都被評估計算之後才會發生。



- 因此四個正反器同時改變其值，如位移暫存器所要求的。此程式碼產生的位移暫存器電路和圖49程式碼所產生的電路相同。
- 考慮將圖50第17到20行的順序相反的影響，如圖51所示。此時程式碼所指定的第一個位移運算，第17行，將w值位移進入最左邊輸出為Q3的正反器。
- 根據程序敘述的語義，對Q3的指定不會發生，直到所有程序內的敘述都被計算評估之後。因此第18行位移現在的Q3值進入輸出為Q2的正反器，在Q3因第17行而改變其值之前。
- 第19和20行將現在的Q2和Q1值分別位移進入輸出為Q1和Q0的正反器。此程式碼產生的電路和圖50敘述順序所產生的電路相同。



另一種四位元位移暫存器程式碼

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
ENTITY shift4 IS
    PORT (
        R          : IN          STD_LOGIC_VECTOR(3 DOWNT0 0) ;
        Clock      : IN          STD_LOGIC ;
        L, w        : IN          STD_LOGIC ;
        Q          : BUFFER      STD_LOGIC_VECTOR(3 DOWNT0 0) ) ;
END shift4 ;

ARCHITECTURE Behavior OF shift4 IS
BEGIN
    PROCESS
    BEGIN
        WAIT UNTIL Clock'EVENT AND Clock = '1' ;
        IF L = '1' THEN
            Q <= R ;
        ELSE
            Q(0) <= Q(1) ;
            Q(1) <= Q(2) ;
            Q(2) <= Q(3) ;
            Q(3) <= w ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖 50四位元位移暫存器的另一種程式碼



敘述順序顛倒的四位元位移暫存器程式碼

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
ENTITY shift4 IS
    PORT (
        R          : IN          STD_LOGIC_VECTOR(3 DOWNT0 0) ;
        Clock      : IN          STD_LOGIC ;
        L, w       : IN          STD_LOGIC ;
        Q          : BUFFER      STD_LOGIC_VECTOR(3 DOWNT0 0) ) ;
END shift4 ;

ARCHITECTURE Behavior OF shift4 IS
BEGIN
    PROCESS
    BEGIN
        WAIT UNTIL Clock'EVENT AND Clock = '1' ;
        IF L = '1' THEN
            Q <= R ;
        ELSE
            Q(3) <= w ;
            Q(2) <= Q(3) ;
            Q(1) <= Q(2) ;
            Q(0) <= Q(1) ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖 51 將圖50 敘述順序顛倒的程式碼



範例九 N位元位移暫存器

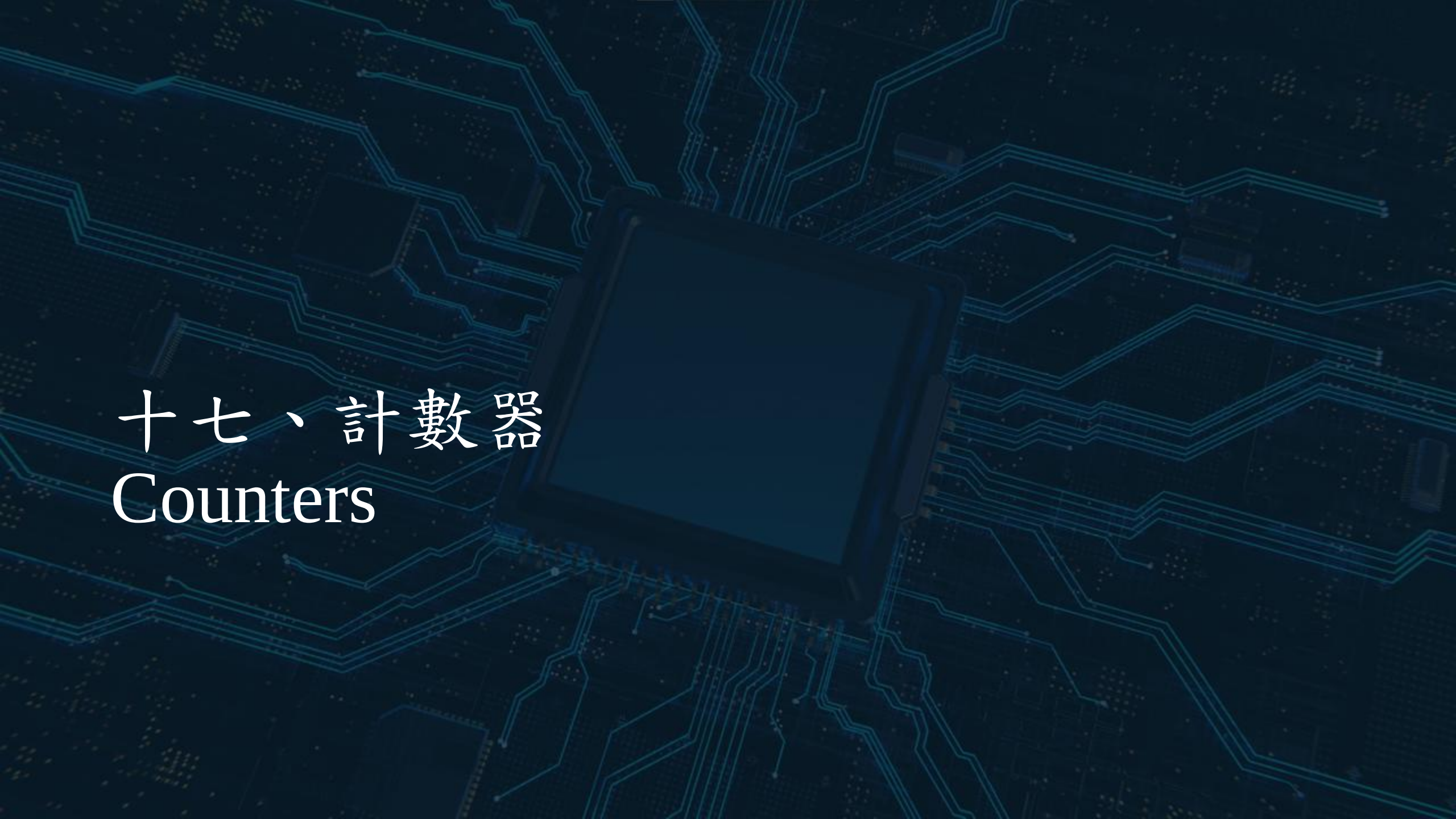
- 圖52的程式碼可用來表示任何尺寸的位移暫存器。**GENERIC**參數**N**設定**正反器的個數**，圖中的預設值為8。此程式碼和圖50大致相同，只有兩個地方不同。
 - 首先，R和Q以N定義。第二，描述位移運算的ELSE敘述被一般化，適用於任何正反器個數。
- 第18到20行分別指定位移運算給最右邊的N - 1個正反器其輸出為QN-2到Q0。使用的架構稱為**FOR LOOP**，類似於**FOR GENERATE**敘述，用來產生一組同作敘述。
- FOR LOOP則是用來產生一組順序敘述。第一次迴圈遞迴將Q1的現在值位移到輸出為Q0的正反器。下一次迴圈遞迴將Q2位移到輸出為Q1的正反器，以此類推，最後一次迴圈遞迴將QN-1位移到輸出為QN-2的正反器。第21行將序列輸入w的值位移到最左邊輸出為QN-1的正反器，完成位移運算。



N位元位移暫存器

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
ENTITY shiftn IS
    GENERIC ( N : INTEGER := 8 );
    PORT (      R          : IN          STD_LOGIC_VECTOR(N-1 DOWNT0 0) ;
            Clock         : IN          STD_LOGIC ;
            L, w          : IN          STD_LOGIC ;
            Q             : BUFFER      STD_LOGIC_VECTOR(N-1 DOWNT0 0) ) ;
END shiftn ;
ARCHITECTURE Behavior OF shiftn IS
BEGIN
    PROCESS
    BEGIN
        WAIT UNTIL Clock'EVENT AND Clock = '1' ;
        IF L = '1' THEN
            Q <= R ;
        ELSE
            Genbits: FOR i IN 0 TO N-2 LOOP
                Q(i) <= Q(i+1) ;
            END LOOP ;
            Q(N-1) <= w ;
        END IF ;
    END PROCESS ;
END Behavior ;
```

圖52 n 位元由左至右位移暫存器的程式碼



十七、計數器

Counters



非同步計數器

Asynchronous Counters



T 正反器組成的上數計數器 Up-counter With T Flip-flops

- 因為向上計數，所以稱為上數計數器 (up- counter)。
- 圖19a的電路為非同步計數器 (asynchronous counter)，或稱為漣波計數器 (ripple counter)。



三位元上數計數器

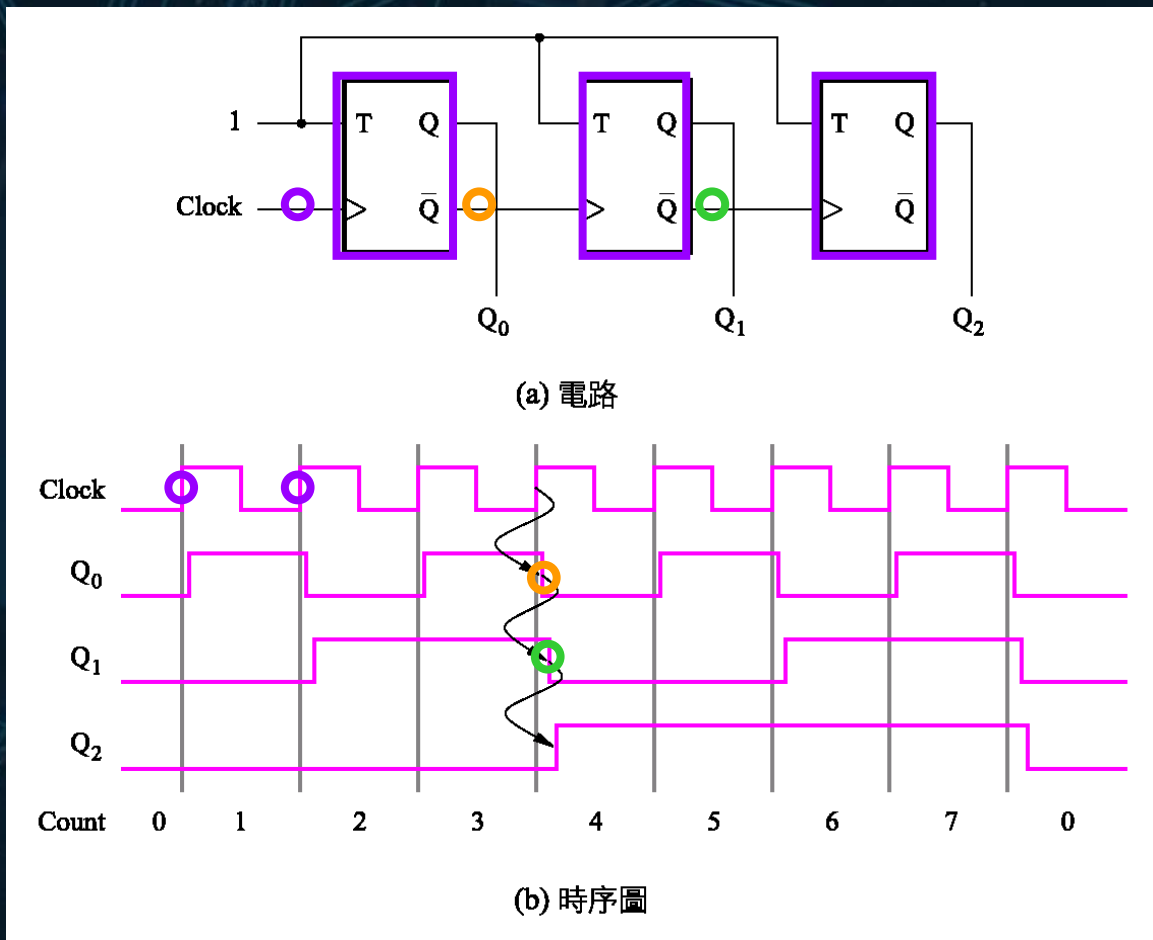


圖19 三位元上數計數器。



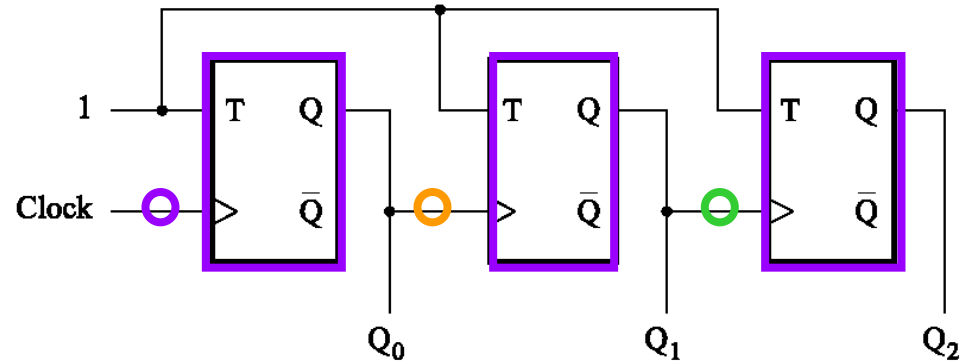
T 正反器組成的下數計數器

Down-counter With T Flip-flops

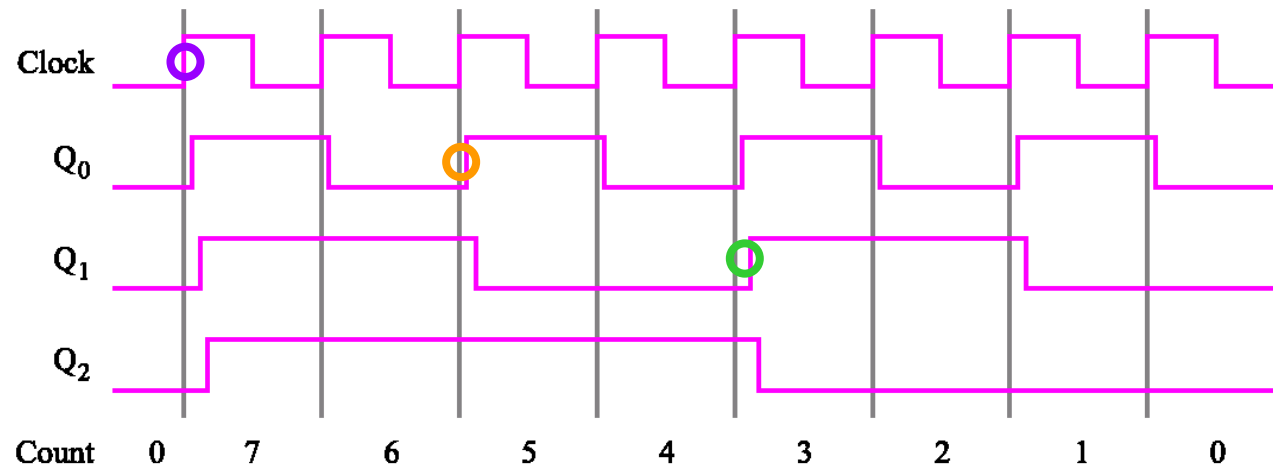
- 因為向下計數，所以稱為下數計數器 (down-counter)。
- 我們可以將圖19a和圖20a的電路功能合併，形成可以上數或下數的計數器。這種計數器稱為上／下數計數器 (up/down-counter)。



三位元下數計數器(非同步(漣波)計數器)



(a) 電路



(b) 時序圖

圖20 三位元下數計數器。



同步計數器

Synchronous Counters



T正反器組成的同步計數器

Synchronous Counter With T Flip-flops

- 若以T正反器實作計數器，則T輸入定義為

$$T_0 = 1$$

$$T_1 = Q_0$$

$$T_2 = Q_0 Q_1$$

$$T_3 = Q_0 Q_1 Q_2$$

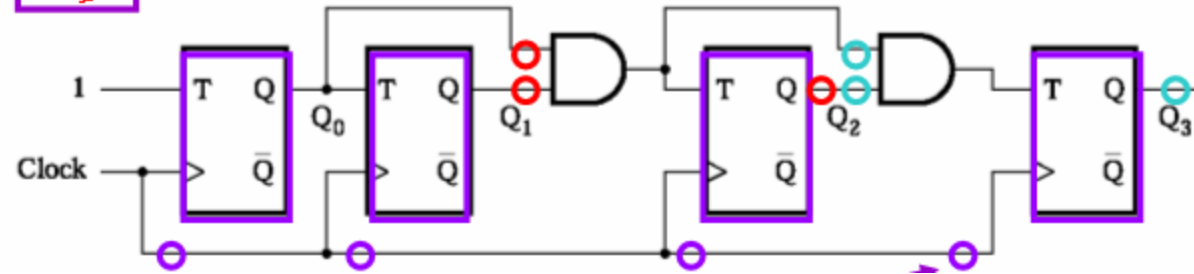
.

.

.

$$T_n = Q_0 Q_1 \cdots Q_{n-1}$$

四位元同步上數計數器



- 以這些式子為基礎，四位元計數器的例子如圖21a所示。
- 圖21b為時序圖，電路行為如同模數16計數器。因為所有改變都發生在Clock訊號有效邊緣同樣的延遲之後，此電路稱為**同步計數器 (synchronous counter)**。

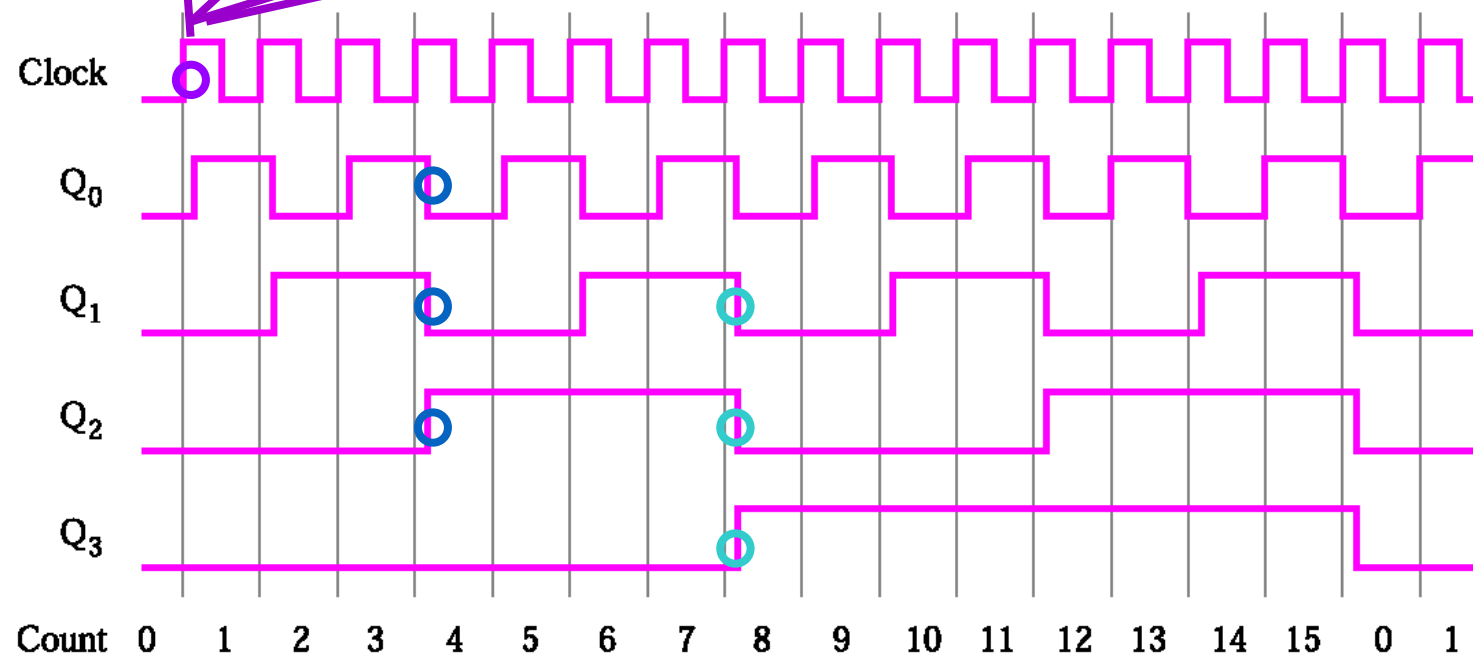
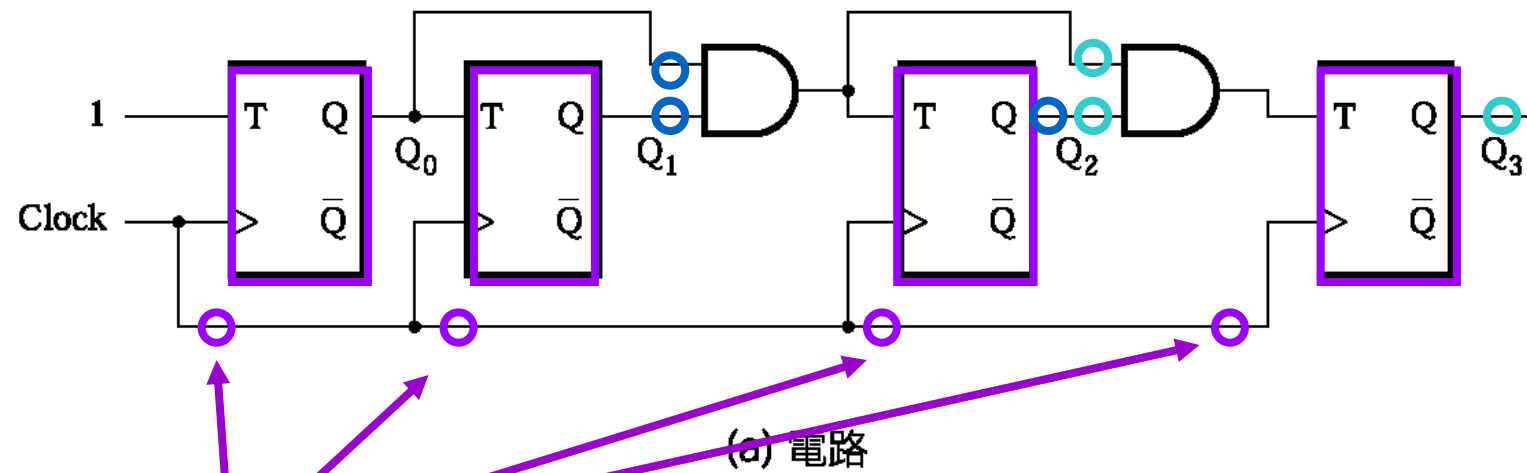


表1 同步上數計數器的推導。

時脈循環	Q ₂	Q ₁	Q ₀
0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8	0	0	0

Q₁ 改變

Q₂ 改變



(b) 時序圖

圖21 四位元同步上數計數器。



致能和清除的能力 Enable And Clear Capability

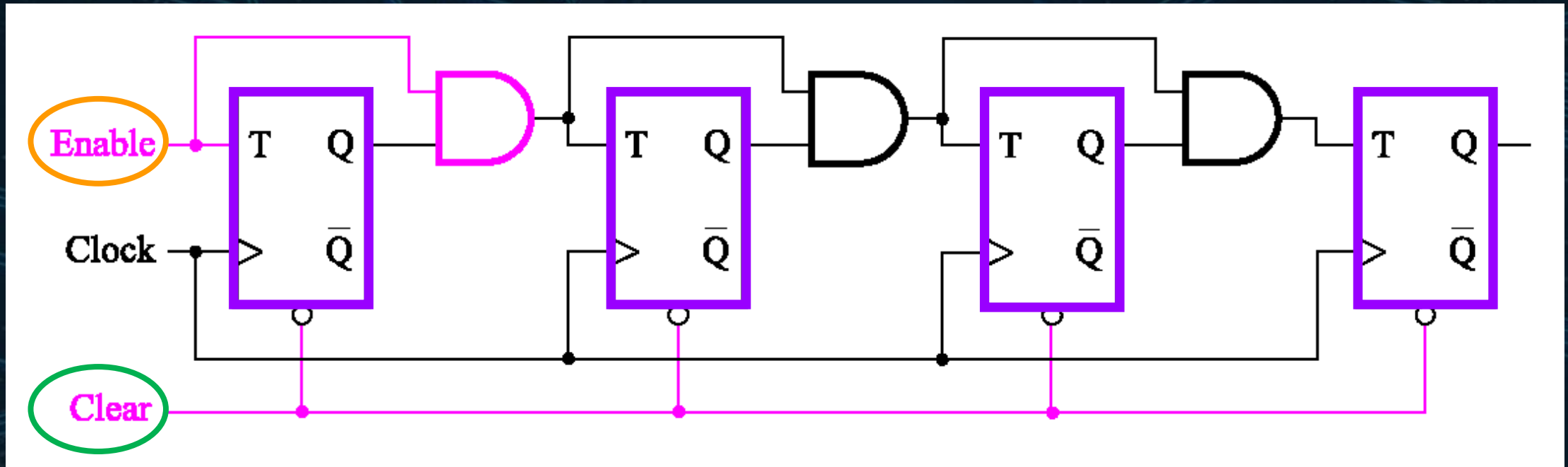


圖22 包含致能和清除的能力。



D正反器組成的同步計數器 Synchronous Counter With D Flip-flops

- 若假設 $Enable = 1$ ，則正反器的 D 輸入由下列表示式所定義

$$D_0 = \bar{Q}_0 = 1 \oplus Q_0$$

$$D_1 = Q_1 \oplus Q_0$$

$$D_2 = Q_2 \oplus Q_1 Q_0$$

$$D_3 = Q_3 \oplus Q_2 Q_1 Q_0$$

- 對較大的計數器，第 i 級定義為

$$D_i = Q_i \oplus Q_{i-1} Q_{i-2} \cdots Q_1 Q_0$$

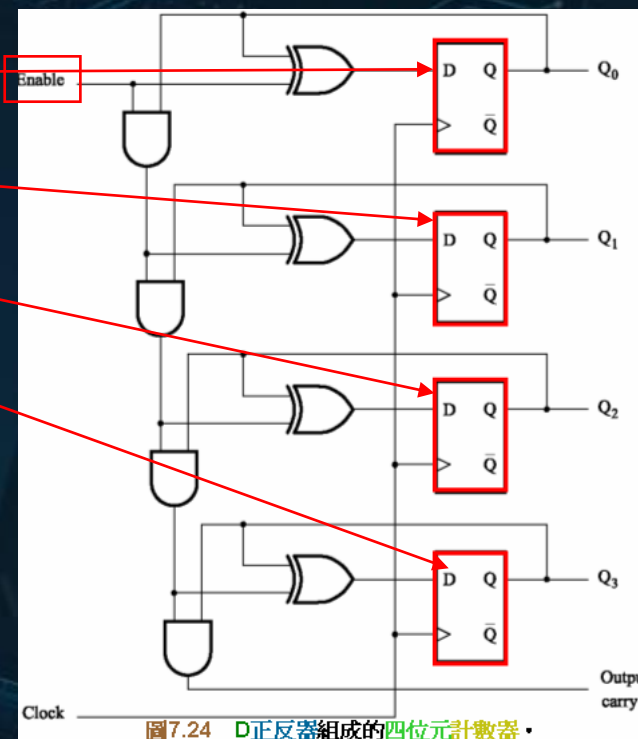


圖 7.24 D 正反器組成的四位元計數器。

- 我們已經包含 $Enable$ 控制訊號，使得計數器只有在 $Enable = 1$ 時計算時脈脈衝個數。實際上，上述方程式被修改如下，以實作圖中的電路



$$D_0 = Q_0 \oplus \text{Enable}$$

$$D_1 = Q_1 \oplus Q_0 \cdot \text{Enable}$$

$$D_2 = Q_2 \oplus Q_1 \cdot Q_0 \cdot \text{Enable}$$

$$D_3 = Q_3 \oplus Q_2 \cdot Q_1 \cdot Q_0 \cdot \text{Enable}$$

- 計數器的運作是以表1的觀察為基礎，只有當所有前級正反器都在狀態 $Q = 1$ ，第 i 級正反器的狀態才會改變。
- 最後注意的是，圖23的計數器基本上和圖22的電路是一樣的。
- T正反器可由D正反器構成，只要提供額外的閘控

$$\begin{aligned} D &= Q\bar{T} + \bar{Q}T \\ &= Q \oplus T \end{aligned}$$

- 因此在圖23的每一級，D正反器和相關的XOR邏輯閘可以實作T正反器的功能。

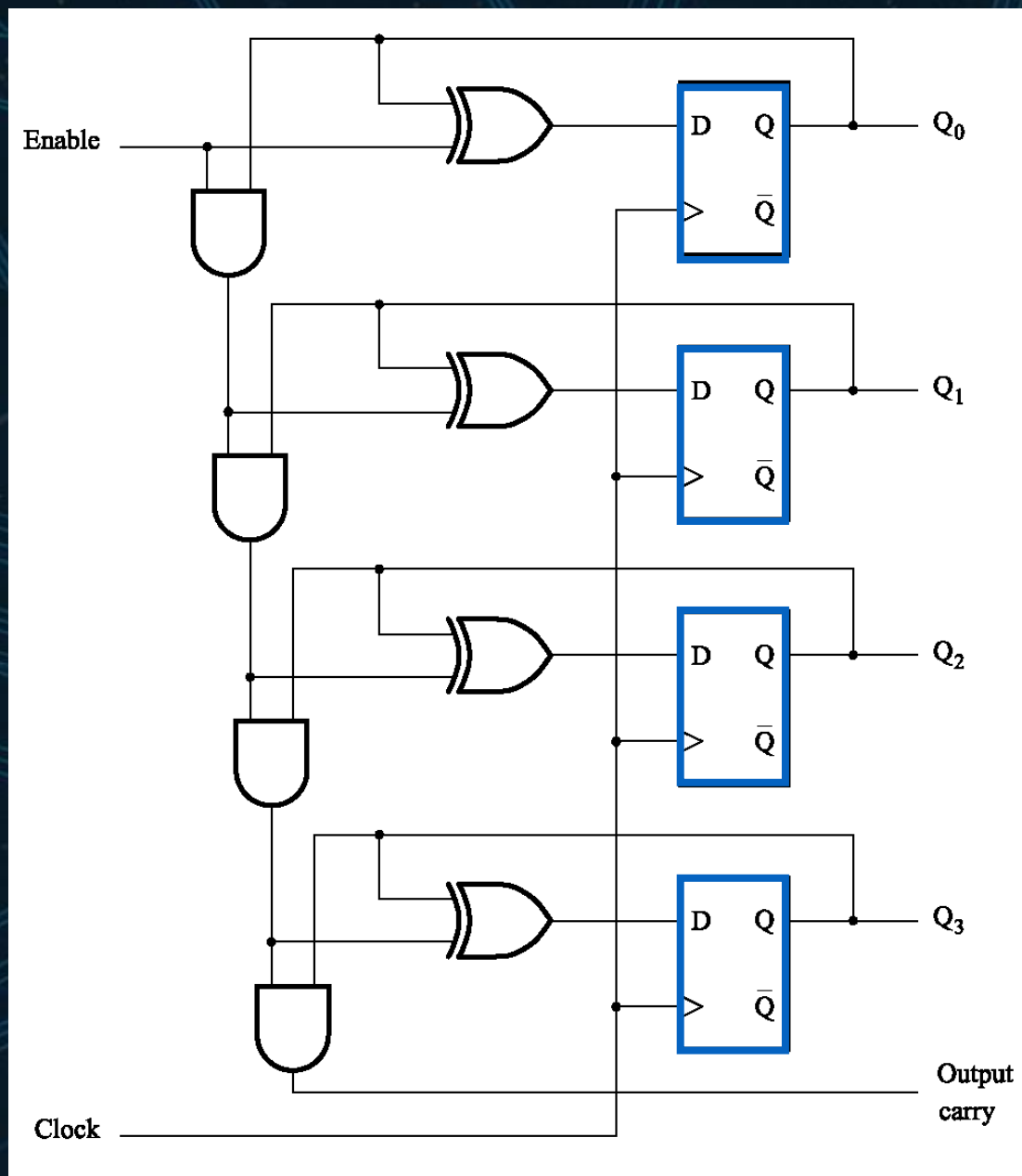


圖23 D正反器組成的四位元計數器。



十八、 平行載入的計數器 Counters With Parallel Load

- 我們常常必須從0開始計數。使用圖22可清除的正反器就能達成。但有時候我們想要從不同的數值開始計數。為此運作模式，計數器電路必須從一些輸入載入初始記數值。
- 圖23的電路可以修改成提供平行載入能力，如圖24所示。在每個D輸入之前安插二輸入多工器。多工器的一個輸入用來提供正常計數運作。另一個輸入為資料位元，可以直接載入正反器。

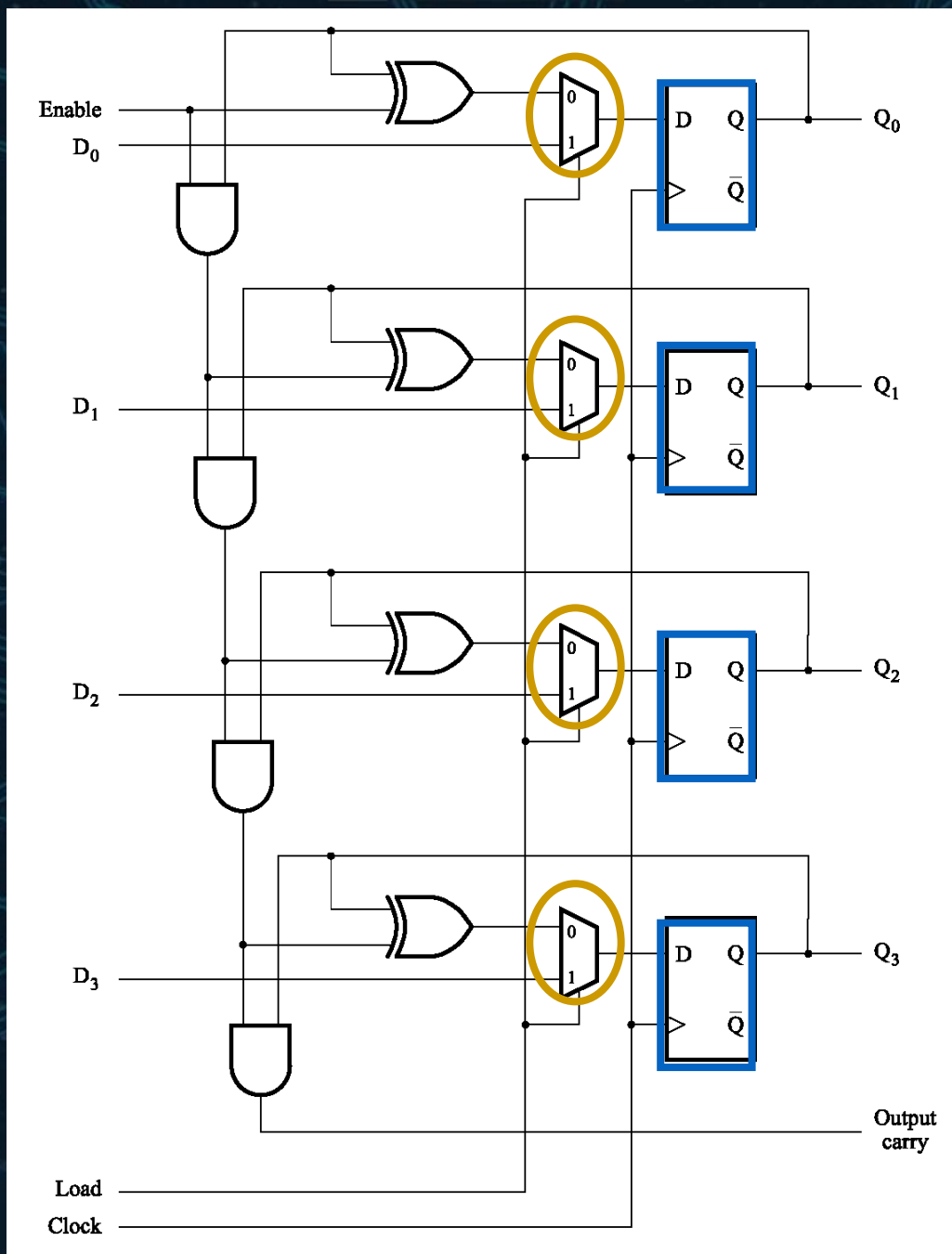
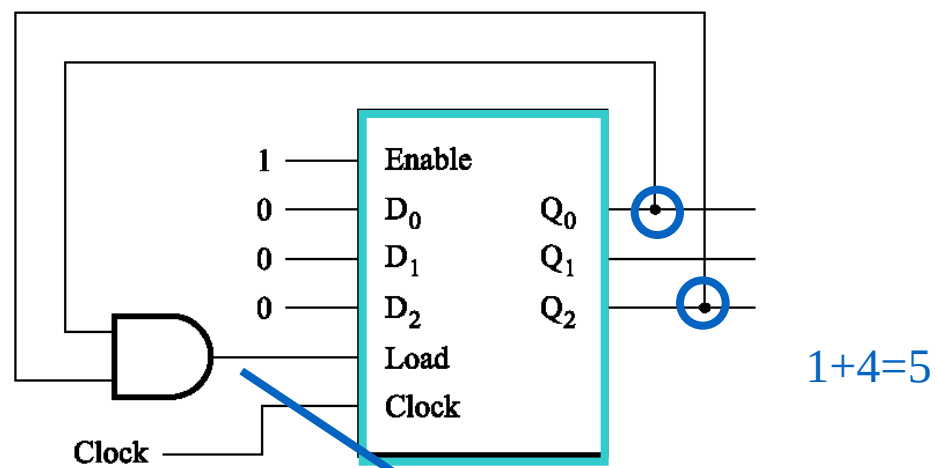


圖24 平行載入的計數器。



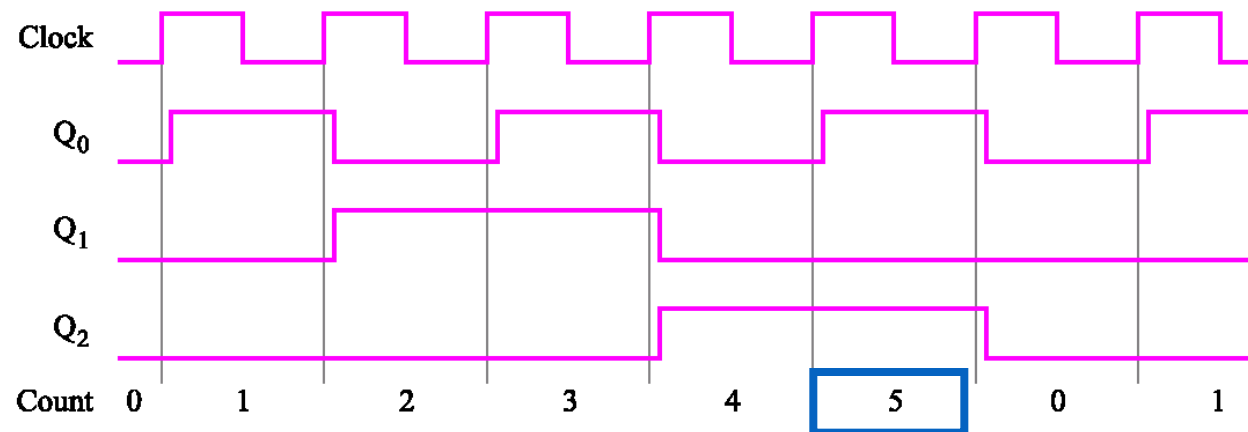
十九、重置同步 Reset Synchronization

- 可以在開始計數之前將計數器清除或重置 (reset) 是相當重要的。
- 因為計數器在時脈有效邊緣被重置，稱這種計數器為**同步重置 (synchronous reset)**。
- 圖26a所用的方法稱為**非同步重置 (asynchronous reset)**。



$$1+4=5$$

(a) 電路



(b) 時序圖

圖25 重置同步的模數6計數器。

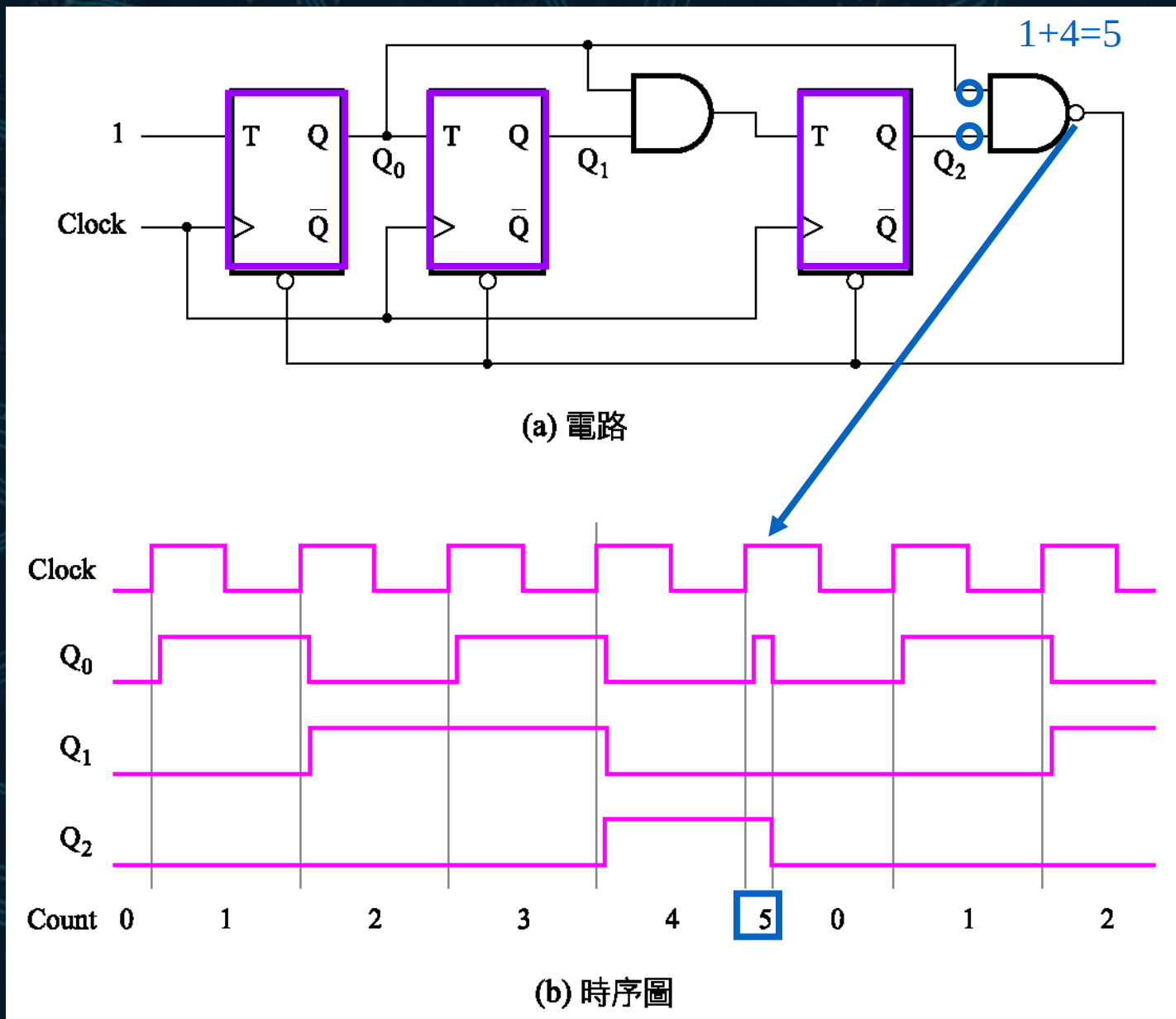
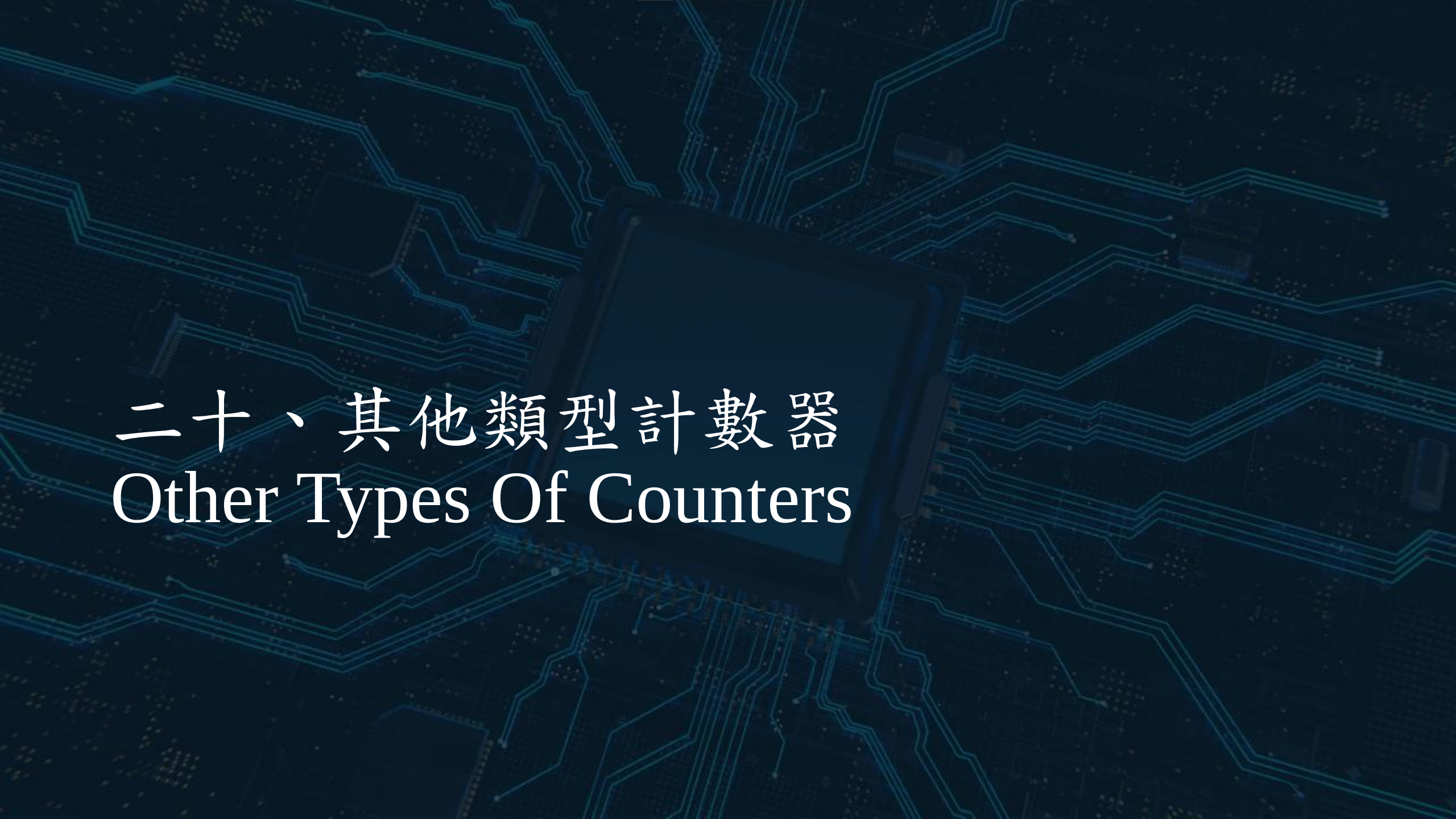


圖26 非同步重置的模數6計數器。



二十、其他類型計數器

Other Types Of Counters



二十一、BCD計數器

BCD Counter

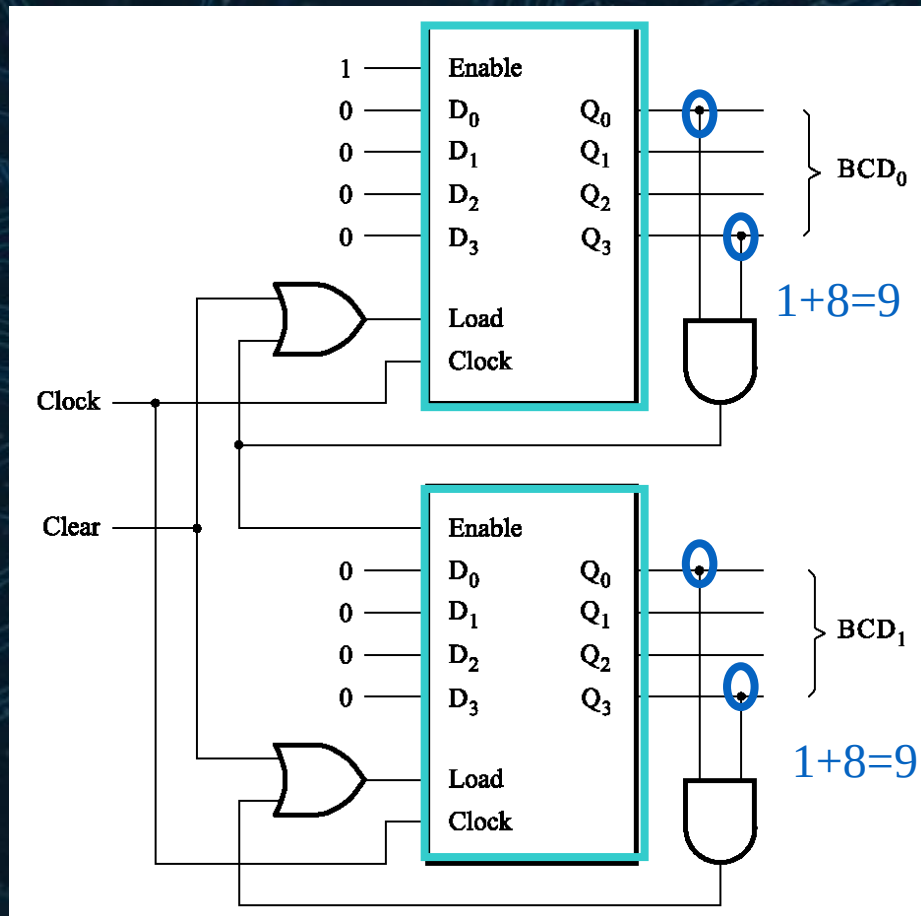


圖27 二數字BCD計數器。



二十二、環狀計數器 Ring Counter

- 圖28a的電路稱為環狀計數器 (Ring Counter)，其運作一開始要在第一級加入一個1。
- 一個1其他為0的編碼方式稱為單一熱門編碼 (One-hot Code)。

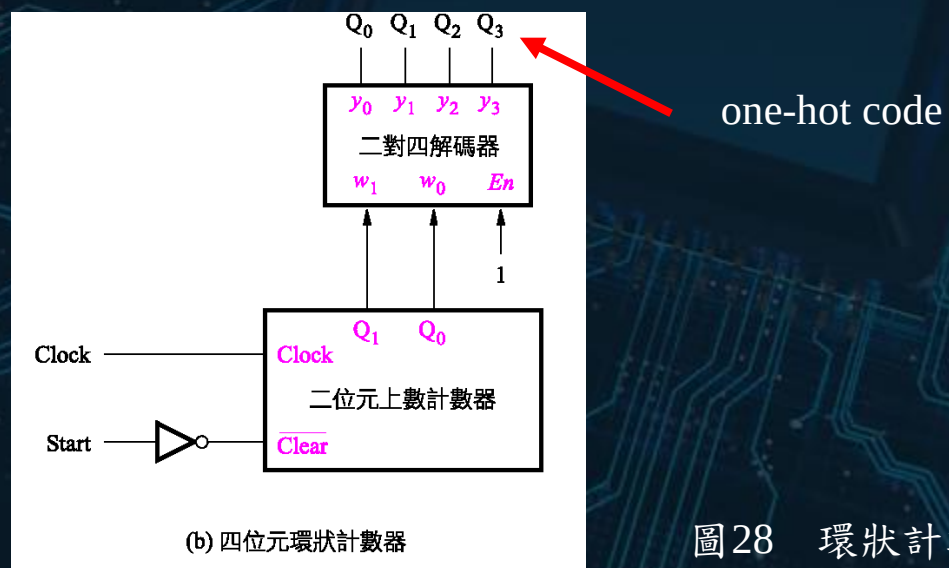
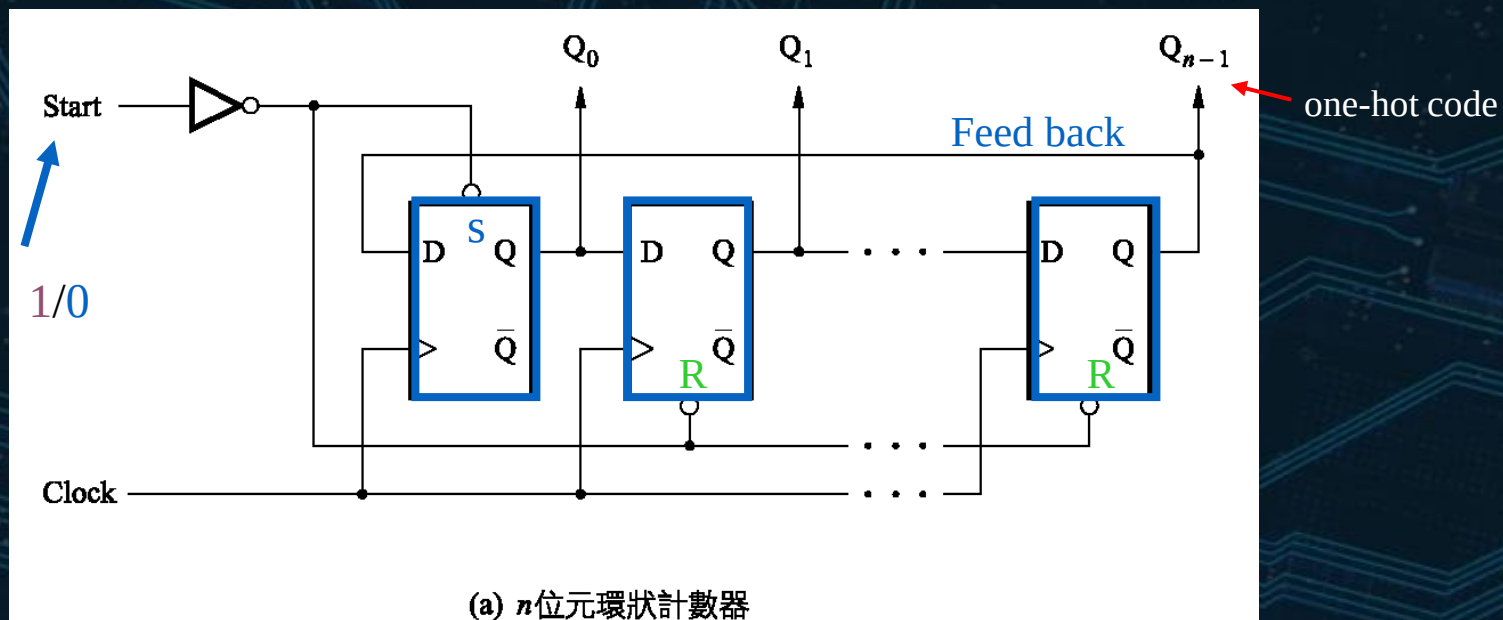


圖28 環狀計數器。



二十三、強生計數器 Johnson Counter

- 若將最後一級的 $\bar{Q}$ 輸出而不是 Q 輸出拉回第一級，可以得到很有趣的環狀計數器變形，如圖所示。此電路稱為強生計數器 (Johnson counter)。

Ex.

0000
1000
1100
1110
1111
0111
0011
0001

一bit不同

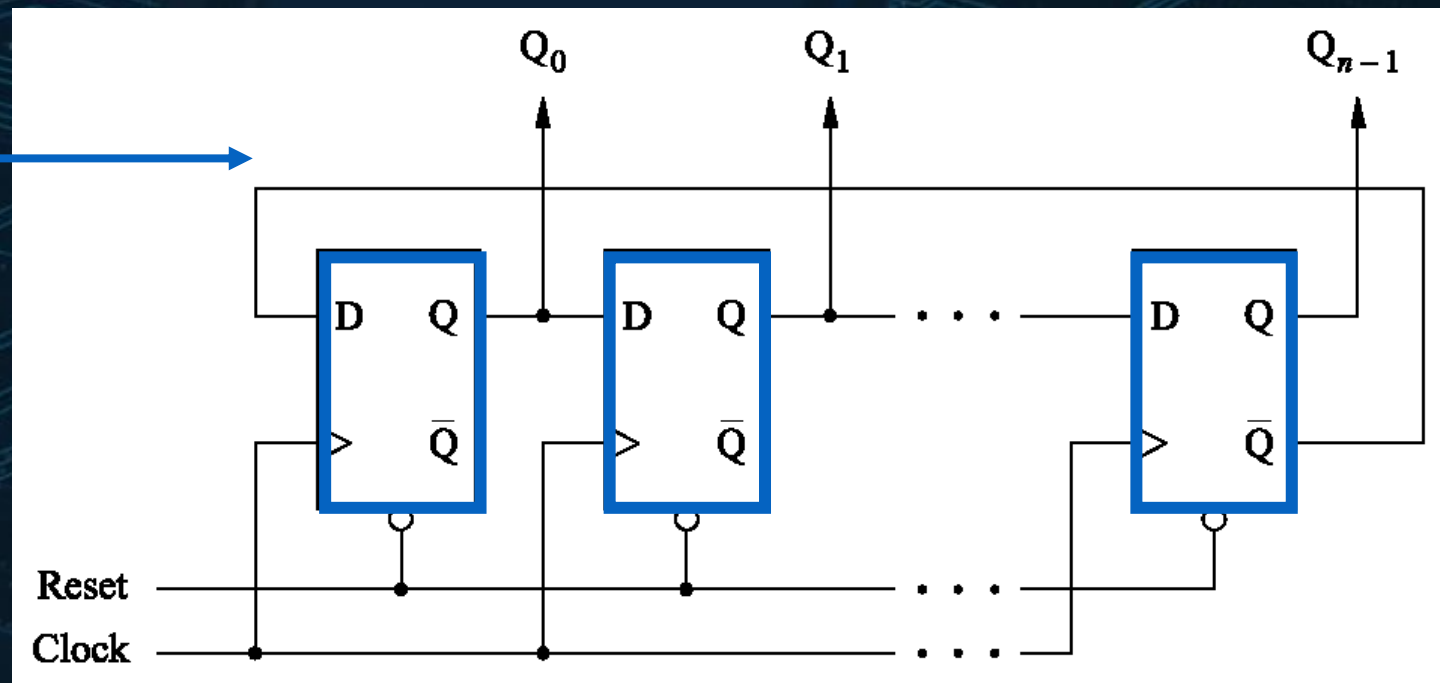


圖29 強生計數器。



範例十 上數計數器

- 圖53為四位元上數計數器的程式碼，其重置輸入為Resetn，致能輸入為E。在architecture中，計數器的正反器以訊號Count表示。
- 程序敘述指定Count的非同步重置，當Resetn = 0時。ELSIF敘述指定在時脈正緣若E = 1，計數加一。若E = 0，程式碼明顯指定Count <= Count。
- 因為隱含的**記憶體**語義，正確描述計數器時此敘述不是必要的，但可以使其意思更為清楚。在程式碼最後Q輸出被指定Count值。
- 若VHDL編譯器使用T正反器，則此程式碼產生圖22的電路，若編譯器選擇D正反器，則產生圖23（加上重置輸入）的電路。



四位元上數計數器

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
USE ieee.std_logic_unsigned.all ;
ENTITY upcount IS
    PORT (    Clock, Resetn, E    : IN        STD_LOGIC ;
            Q                                : OUT    STD_LOGIC_VECTOR (3 DOWNT0 0));
END upcount ;

ARCHITECTURE Behavior OF upcount IS
    SIGNAL Count : STD_LOGIC_VECTOR (3 DOWNT0 0) ;
BEGIN
    PROCESS ( Clock, Resetn )
    BEGIN
        IF Resetn = '0' THEN
            Count <= "0000" ;
        ELSIF (Clock'EVENT AND Clock = '1' ) THEN
            IF E = '1' THEN
                Count <= Count + 1 ;
            ELSE
                Count <= Count ;
            END IF ;
        END IF ;
    END PROCESS ;
    Q <= Count ;
END Behavior ;
```

圖53 四位元上數計數器的程式碼



致能和清除的能力 Enable And Clear Capability

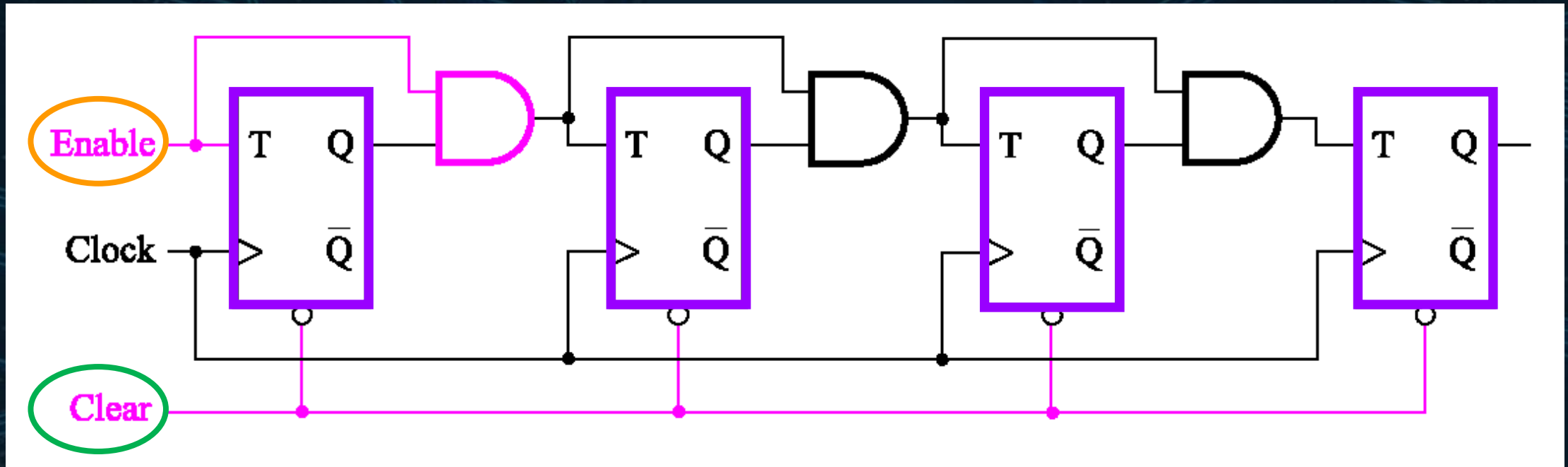


圖22 包含致能和清除的能力。

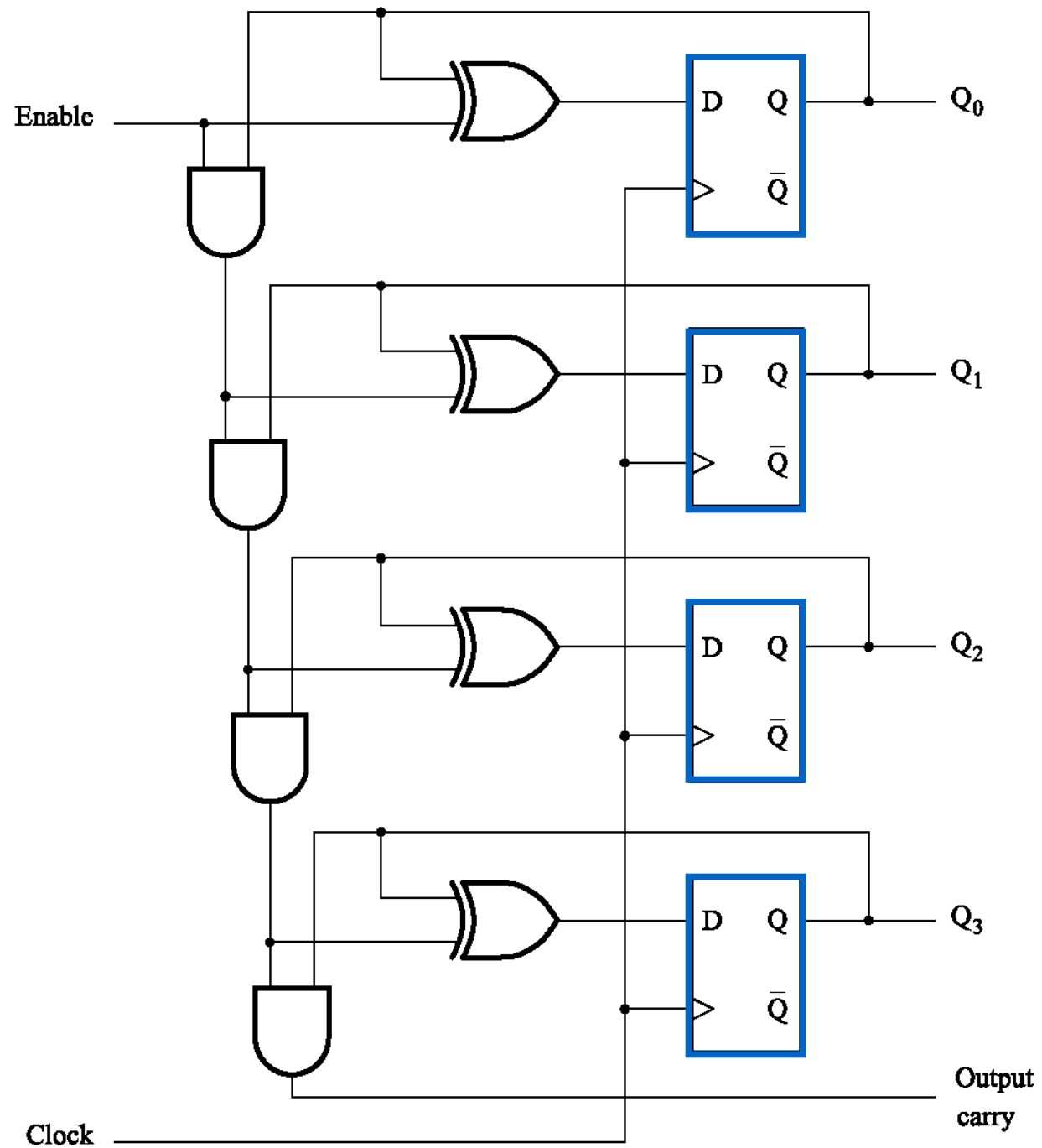


圖23 D正反器組成的四位元計數器。



範例十一

具有平行載入輸入和重置輸入的上數計數器

- 計數器通常以**INTEGER**類型定義於VHDL，圖54的程式碼定義一個**具有平行載入輸入和重置輸入的上數計數器**。平行資料R和計數器輸出Q是以INTEGER類型定義。
- 因為其範圍是0到15，這些訊號都代表四位元的量。圖53中訊號Count是定義來表示計數器中的正反器。若Q輸出為BUFFER模式則不需要這些訊號。
- 如圖54所示。程序開頭的if-then-else敘述包含和圖53相同的非同步重置。ELSIF敘述指定在時脈正緣若 $L = 1$ ，從R輸入平行載入計數器的正反器。若 $L = 0$ ，則計數加一。程式碼所代表的電路如圖24所示。

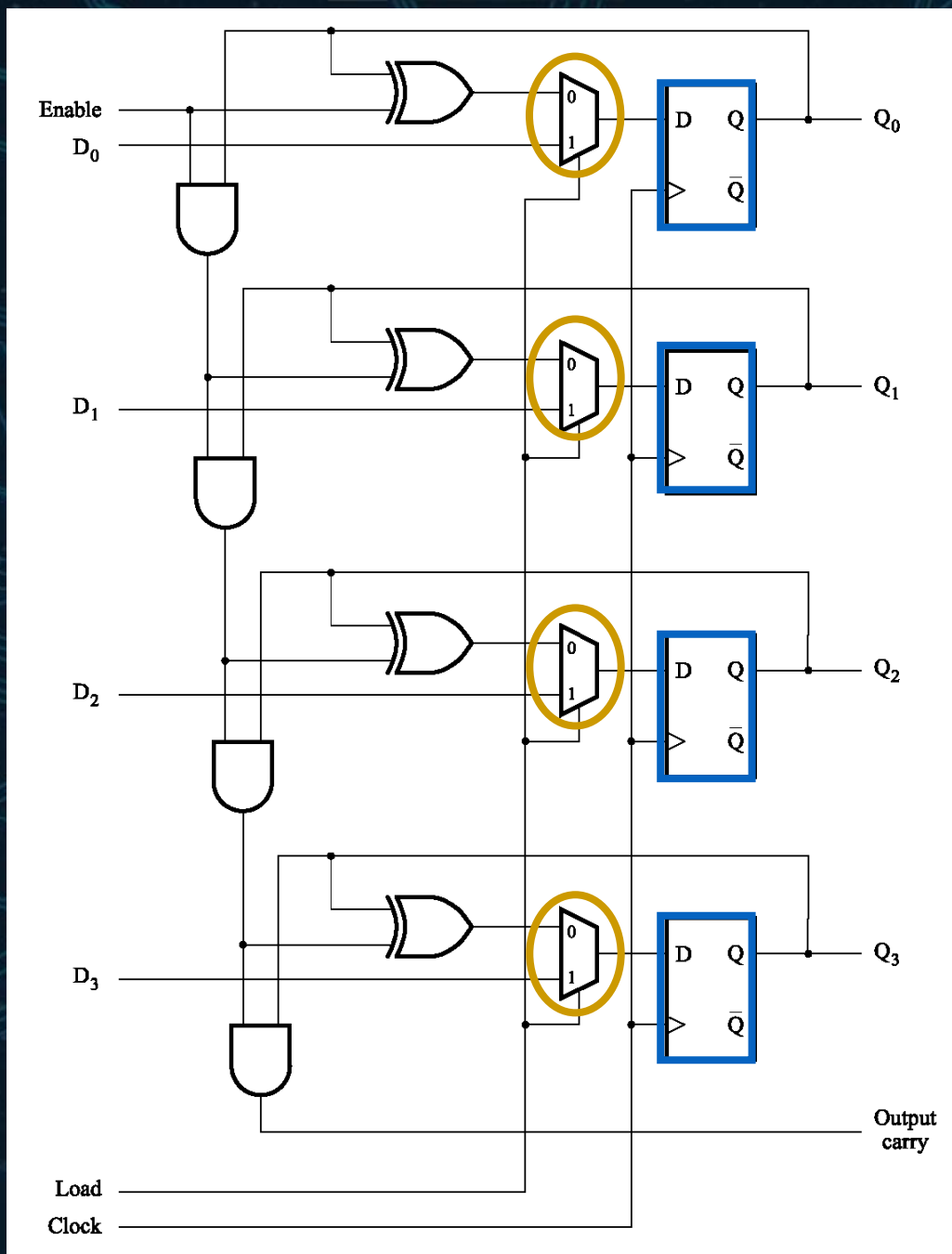


圖24 平行載入的計數器。



具有平行載入輸入和重置輸入的上數計數器

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;

ENTITY upcount IS
    PORT (    R          : IN          INTEGER RANGE 0 TO 15 ;
            Clock, Resetn, L : IN          STD_LOGIC ;
            Q              : BUFFER    INTEGER RANGE 0 TO 15 )
;
END upcount ;

ARCHITECTURE Behavior OF upcount IS
BEGIN
    PROCESS ( Clock, Resetn )
    BEGIN
        IF Resetn = '0' THEN
            Q <= 0 ;
        ELSIF (Clock'EVENT AND Clock = '1') THEN
            IF L = '1' THEN
                Q <= R ;
            ELSE
                Q <= Q + 1 ;
            END IF;
        END IF;
    END PROCESS;
END Behavior;
```

圖 54 具有平行載入輸入的四位元計數器，使用INTEGER 訊號



範例十二 下數計數器

- 下數計數器 圖55為下數計數器`downcnt`的程式碼。為使其容易改變起始計數，定義為**GENERIC**參數`modulus`。
- 在時脈正緣若 $L = 1$ ，計數器載入值 $modulus - 1$ ，若 $L = 0$ ，則計數減一。計數器也包含致能輸入 E 。設定 $E = 0$ 可避免**有效時脈邊緣發生時正反器內容改變**。



下數計數器

```
LIBRARY ieee ;
USE ieee.std_logic_1164.all ;
ENTITY downcnt IS
    GENERIC ( modulus : INTEGER := 8 ) ;
    PORT (   Clock, L, E   : IN   STD_LOGIC ;
            Q               : OUT  INTEGER RANGE 0 TO modulus-1 ) ;
END downcnt ;

ARCHITECTURE Behavior OF downcnt IS
    SIGNAL Count : INTEGER RANGE 0 TO modulus-1 ;
BEGIN
    PROCESS
    BEGIN
        WAIT UNTIL (Clock'EVENT AND Clock = '1' ) ;
        IF E = '1' THEN
            IF L = '1' THEN
                Count <= modulus-1 ;
            ELSE
                Count <= Count-1 ;
            END IF ;
        END IF ;
    END PROCESS;
    Q <= Count ;
END Behavior ;
```

圖 55下數計數器的程式碼